

---

## NiCE Search Grid Programmer's Guide Version 20.1.2.1

---

---

### Programmer's Introduction to NiCE Search Grid

---

---

#### Basic Concepts

---

---

##### Phonetic Search

---

Search Grid's phonetic engine searches recorded speech for phonemes, which are the smallest fragments of sound that distinguish one word from another. The word *cat* has three phonemes, which in this case map to the letters that spell it: the k sound (spelled with a c), the æ sound (spelled with an a), and the final t sound. The word *that* also has three phonemes: the last two are the same as in *cat*, but the first is the sound spelled by the two-letter combination *th*. It is the difference between the phonemes k and th that make *cat* and *that* two distinct words. Similarly, the word *Kate* is distinguished from *cat* by the substitution of the phoneme ey (the long a sound) for æ.

The phonetic engine finds phonemes using a model programmed to match specific characteristics of an acoustic waveform with specific phonemes on a probabilistic basis. Neither the model nor the phonetic engine has any notion of language or non-language, nor do they have any of the considerable aids to speech perception wired into the human brain. The phonetic engine therefore considers all characteristics of a waveform equally; it cannot differentiate characteristics caused by speech from those caused by the device that recorded that speech.

These two factors each contribute significantly to the way the model "learns" about phonemes:

- the acoustics of the device used to record the speech, and
- the linguistic characteristics of the speakers.

---

##### Device Acoustics

---

The device used to record speech shapes the waveform in significant ways. Microphones from different manufacturers have different frequency responses, which can cause significant alterations in how a waveform is recorded. Some devices may introduce distortion, such as clipping or reverberation.

In addition, the circumstances under which the speech was recorded can also alter the waveform. A recording might

include background noise, such as other people speaking faintly, music, or machinery. Recordings of telephone interactions often include dial tones or tones from pressing the telephone buttons.

Speech recorded in a quiet studio with a broadcast-quality microphone and other professional equipment is going to produce a different waveform from the same speech recorded through a telephone in a laundromat. For this reason, the phonetic engine is developed based on two different models: one for broadcast, and one for telephony. As expected, the phonetic engine recognizes speech more accurately when the audio has a minimum of background noise and distortion.

---

## Speakers

The way people speak varies at least as much as the acoustics of a given set of devices. People in broadcast studios tend to speak differently from people in laundromats shouting into telephones; rehearsed speech has different rhythms and intonation from spontaneous speech.

Similarly, the same words will produce a different waveform when spoken by a middle-aged man from Portland, Maine; a young woman from Huntsville, Alabama; or a child from Los Angeles, California. Even the same speaker saying the same thing will produce a different waveform when suffering from a head cold.

For this reason, the phonetic engine is "trained" on a variety of representative speakers of both genders, various ages, and from various dialect regions.

As expected, the phonetic engine recognizes speech more accurately when the speaker is speaking clearly with an accent well represented in the training data.

---

## Language Packs

A *language pack* is a collection of phonetic models that instruct the phonetic engine how to handle a specific combination of spoken language and device acoustics. For example, one language may support North American English recorded with 8kHz telephony codecs, while another language pack may support Latin American Spanish recorded with 16kHz broadcast codecs. A Search Grid instance may have one or more language packs installed. From the programmer's perspective, language packs are important because selecting a language pack influences the phonetic indexes that Search Grid creates, which in turn influence the quality of the search.

---

## Phonetic Indexes

Creating a phonetic index is the process that makes speech searchable. All audio data must be indexed before a user can search it. There are two main types of phonetic index: the *pat*, which is required for all types of phonetic search; and the phonetic digest, which is used for searching large volumes of media.

Most client applications do not need to interact directly with phonetic indexes; it is sufficient for the client application to request that Search Grid create a *pat* for each media file to be searched. Search Grid will then queue the request, distribute the indexing load across the grid, save the *pat* to persistent storage, and create a phonetic digest. Similarly, client applications do not need to interact directly with phonetic indexes to perform a phonetic search. Search Grid will route the search to the appropriate server or servers, load the applicable phonetic index from storage, and attempt to

cache the index in memory.



Search Grid never stores audio data; Search Grid stores only the phonetic indexes that it creates or that a client application uploads.

---

## Searching

To search, you specify a term to search for (the search term) as well as a set of optional parameters that refine the search algorithm. The phonetic search engine looks for a sequence of phonemes in the audio data that matches the sequence of phonemes comprising the search term. Searching occurs faster than indexing and many times faster than real time. When searching is complete, the system returns the results to the client application

The search term can be any combination of dictionary words or names (such as Albert Einstein, London Bridge, or Waukegan, Illinois); you can also spell out a term according to how it sounds, or by entering phonemes directly. The phonetic engine converts your search term to phonemes (if necessary), and then searches the phonetic indexes for the specified sequence of phonemes.

You can also specify that a given word or phrase must occur within a certain time span of another word or phrase.

Positive search results, or *hits*, include several pieces of information:

- a reference to the media that contains the hit
- a reference to the search term associated with the hit
- a time offset from the start of the media that locates the hit
- a time offset that approximately locates the corresponding end of the hit
- a confidence score

---

## Confidence Scores

The phonetic search engine does not make definite decisions; rather, it works on a probabilistic basis. The confidence score, or simply *score*, is the number it uses to express how well the search term matches a given audio fragment in the target track. This is a number between 0 and 100 that represents the similarity between the search term and the specific fragment of audio that was found.

Search Grid allows client applications to set a threshold that limits results to those with a confidence score above the specified threshold value. However, choosing a threshold can be challenging because audio quality, speaker characteristics, and the search term itself all have an effect on confidence score values. As a general rule, a hit with a confidence score below 60 is a poor match; that is, a hit with a low probability of being correct. Similarly, a hit with a confidence score above 90 has a high probability of being correct.

---

## Autothresholding

The difficulties of choosing a score threshold can be avoided by using Search Grid's autothresholding feature. When autothresholding, the phonetic engine automatically chooses an appropriate threshold by analyzing the audio quality, speaker characteristics, and search term.

Although autothresholding is easy to use, its probabilistic nature can cause results to fluctuate over time, even between two searches for the same term across the same media. Applications that require absolutely consistent results should explicitly specify a confidence score threshold; applications may cache previously calculated autothreshold values for this purpose.

Another way to create consistent autothreshold results is to use *invariant data*. Each language pack includes a set of invariant data (also called "normalization data" or "default normalization data") that can be used for autothresholding. Another advantage of using invariant data is that it permits autothresholding before Search Grid has created any pats. However, an autothreshold value calculated against invariant data will produce less accurate results (that is, will be less likely to produce the specified false alarm rate) than a threshold calculated from client-supplied media.

---

## Search Algorithms

Search Grid supports two distinct search algorithms, conventional search and large scale search (LSS). The two algorithms have very different performance, resource usage, and search quality characteristics. Conventional search and LSS also differ in their availability. All Search Grid instances support conventional search, but LSS can be disabled to conserve the additional resources that it consumes. Consult your Search Grid administrator to determine whether LSS is enabled for your instance. But granted that LSS is enabled, client applications can use one or both search algorithms in the same Search Grid instance depending on the business need. The following table summarizes the differences between conventional search and LSS, and subsequent sections give more detail on each characteristic:

| Characteristic       | Conventional Search | Large Scale Search |
|----------------------|---------------------|--------------------|
| Indexing Performance | Higher              | Lower              |
| Storage Space        | Less                | More               |
| Recall               | Lower               | Higher             |
| Repeatable Search    | Yes                 | No                 |
| Structured Queries   | Yes                 | No                 |
| Search Terms         | Yes                 | Yes                |

| Characteristic      | Conventional Search | Large Scale Search |
|---------------------|---------------------|--------------------|
| Search Performance  | Lower               | Higher             |
| Performance Control | Fixed               | Variable           |

## Indexing Performance and Storage Space

LSS uses phonetic indexes (specifically, phonetic digests) that conventional search does not use. These additional artifacts require CPU time to create and storage space to preserve. As a result, a Search Grid instance with LSS enabled spends twice as much CPU time creating phonetic indexes and consumes about 150% more storage space than if LSS were disabled.

## Search Predicates

A *search predicate* represents the phonetic artifact to find in a phonetic index. The simplest search predicate is a term, which is supported by both conventional search and LSS. Conventional search is the only mechanism that can support structured queries, the more sophisticated types of search predicates that are described below.

## Recall

Recall refers to the degree to which a search returns all the available matches. Conventional search has higher recall than LSS, so a conventional search is more likely to return all the available utterances of a given search term. This makes conventional search a better choice when finding the maximum possible results is important; one example is regulatory compliance enforcement. In contrast, LSS can be a better choice when the goal is to find any matches; an example is targeted listening, when a user needs to find example audio for further analysis.

## Repeatable Search

Multiple conventional searches for the same terms over the same media with the same options are guaranteed to produce identical search results. (This, of course, assumes no application upgrades or system errors occur between the searches.) In contrast, repeating the same LSS search may return slightly different results. This is a consequence of Search Grid periodically reorganizing its phonetic digests to maintain performance and conserve storage space.

## Search Performance and Performance Control

When searching over a large volume (hundreds or thousands of hours) of media, LSS is several orders of magnitude faster than conventional search. LSS also allows the client application to make tradeoffs between search performance and recall. Increasing the value of the candidates per hour (CPH) setting causes the LSS algorithm to look for more results at the expense of decreasing search performance. In contrast, conventional search performance is almost entirely a function of the duration of media being searched.

## Search Term Validation

Each language pack has its own set of rules for what comprises a valid search term. For example, most language packs disallow numerals and non-alphanumeric characters like @ or \$. To help shield end users from these details, Search Grid allows client applications to validate search terms before executing a search.

If a search term is invalid, Search Grid will attempt to *normalize* the term to produce a valid term. For example, in a North American English pack, the term "\$1000" will normalize to "one thousand dollars". Note that not all language packs support text normalization; consult the language pack's documentation to be sure.

If the search term is valid, Search Grid will attempt to produce a *phonetic respelling* of the term. A phonetic respelling indicates the pronunciation of the word for which Search Grid will search. For example, a phonetic respelling of "read" with a North American English pack might be "RED". A client application can present the respelling to the user to verify the pronunciation is what the user intended. Here are some tips for using phonetic respelling:

- Respelled terms are intended for presentation to a user, not for replacing the original search term. Searching for a respelled version of a term may result in reduced search quality compared to searching for the original term.
- Not all language packs support phonetic respelling; consult the language pack's documentation to be sure.
- Some valid search terms—usually nonwords like "th"—do not produce a phonetic respelling even if the language pack supports the feature.

---

## Classification

Classifying audio entails searching the media for several types of content—some of them non-phonetic—to produce *segments* describing regions in the audio where the content was found. The segments produced by classification may be of direct interest to the client application, and they may be used to filter search results.

Like pat creation, classification requires Search Grid to read the media file.

Classification can identify multiple types of audio content:

- speech
- DTMF tones
- speaker gender

See the [Segments and Classifiers](#) section for more information on specific classification functions.

---

## Language Identification

Many environments produce media with different spoken languages. For example, call center recordings might contain some conversations in English and others in Spanish. In such environments, the language identification feature can detect which language is used in each recorded call.

Language identification is a useful step in deciding what language pack to use for pat creation and subsequent searching. However, language is not the only factor in choosing the ideal language pack. Language pack choice should also consider acoustic properties such as sample rate and codec, which are available in each pack's manifest.

Language identification is intended for media containing a single spoken language. If multiple languages are spoken in a single piece of media, language identification results will be less accurate.

Language identification depends on a *language identification model* that defines the set of languages that can be identified. Search Grid ships with two models for demonstration purposes. In most cases, NiCE Professional Services will need to create a custom model for a specific customer environment. The custom model will yield much higher accuracy than the included demonstration models.

See the [Languages for Language Identification](#) section for more information on using language identification.

---

## Phrase Recognition

In phonetic search, the user asks the application to find utterances of a specific set of terms. But sometimes it can be a challenge to know what to search for in the first place. Phrase recognition can help answer this need by converting recorded speech to textual information. The resulting text can then be imported into a text analytics system for further processing.

Phrase recognition differs from phonetic search in several important ways. First, phrase recognition is computationally expensive: Compared to creating a pat, phrase recognition takes two orders of magnitude more CPU time and hundreds more megabytes of memory.

Second, phrase recognition depends on using a *phrase pack* rather than a language pack. Phrase packs have narrower applicability than language packs. For example, the same North American English language pack could be as effective in a banking call center as in a hotel call center. But banking and hotel call centers would need distinct phrase packs to get high quality phrase recognition results. A phrase pack produced for a specific company could produce even better results. For this reason, phrase packs in Search Grid are tenant-specific, not a global resource like language packs.

---

## Structured Queries

A structured query is an advanced technique for searching multiple terms with complex constraints. The process of creating an effective structured query is quite involved, and usually requires special training. For more information about structured queries, see [Structured Queries](#) section.

---

## Required Client Application Functions

---

### Track Media Lifecycle

The most basic job of the client application is to inform Search Grid of what media exist to be searched; this is called *declaring* the media. The client application must also inform Search Grid when media is no longer relevant to the system; this is called *dropping* the media. In some applications, the media to be searched is a static set, so the client can perform a one-time initialization routine to inform Search Grid of all the available media. But more often, client applications incorporate media in a workflow context, with new media being periodically introduced and old media being removed. The client application must keep the Search Grid instance in sync with the changes to the set of relevant

media.

## Related API Operations

- [Declare Media](#)
- [Drop Media](#)

---

## Manage Media Files

To create phonetic indexes, Search Grid must access media files. For some client applications, this is a simple matter of informing Search Grid of the location of existing media on a file system or web server. In other environments, client applications need to take a more active role to ensure the media is accessible and usable by Search Grid.

For example, consider an enterprise that stores media in a custom content management system with a proprietary access API. Search Grid services cannot use the proprietary API, so a client application would need to provide other means for Search Grid to access the media. One solution would involve *proxy files*, temporary file system copies of the media created specifically for Search Grid to read. The client application would create the proxy files and then delete them as Search Grid completes the requested media operations.

An alternative solution would be for the client application to include a web server that acts as a front end to the content management system. The client would provide Search Grid with URIs pointing to the web server. As Search Grid requests media from the web server, the web server would retrieve the media from the content management system and return it to Search Grid. (See the [Web Server Media Locations](#) section for more information.)

In addition to managing the media files themselves, the client application must establish which logical contents of the media files should be processed; specifically, the client must choose the appropriate track and channel. A *track* is a group of audio streams suited for a common purpose, and usually meant to be heard together. Media files with multiple tracks are commonly used in video production for alternative soundtracks such as English, Spanish, and Commentary; multi-track files are rarely seen in telephony environments. A *channel* is a single audio stream usually meant to play through a specific speaker. *Right* and *left* channels are common examples; video production often involves additional channels such as *center*, *left surround*, and so on.

By default, Search Grid processes all channels in a media file's first track, but it may be beneficial to give Search Grid more specific instructions. For example, a call center application may instruct Search Grid to process the left and right channels separately because one represents the call center agent and the other represents a customer. A video production application may want to process only the center channel of the Spanish track.

Tracks and channels are referenced by numeric identifiers. Channel and (especially) track identifier semantics vary across media file formats and the applications that created them, so part of planning a Search Grid deployment should involve establishing which identifier values refer to the tracks and channels of interest. As a starting point, in most stereo audio files, channel zero is left, and channel one is right. There is less standardization across multi-track audio files, but track zero is often the "primary" track.



## Related API Operations

- [Request Media Operation](#)

---

## Track Search Grid Media IDs

Most client applications will have some form of unique identifier for the media, e.g. a GUID, a database ID, a URI, or a filename. In Search Grid terminology, this identifier is called the *external media ID*, where *external* means "external to Search Grid". Commonly, the structure of an existing system's unique media identifier was not designed for the purpose of large-scale operations like phonetic search. For this reason, Search Grid assigns its own unique media ID to each piece of media when it is declared. Most Search Grid operations that reference media, including searches, use this media ID rather than the external media ID to identify media.



As of Search Grid 3.2, Search Grid's media ID has been widened from a 32-bit integer value to a 64-bit long integer value and can be any positive value up to  $2^{63} - 1$  (that is 9,223,372,036,854,775,807). Client applications **must** accommodate the full range of possible values.

Client applications should record Search Grid's media ID in persistent storage and use this information to translate back and forth between the external media ID and Search Grid's media ID as needed. (Search Grid also offers operations to translate between its own media IDs and external media IDs; however, these queries may not have sufficient performance characteristics to support interactive use.)

The Media Declaration Availability section describes additional client considerations for tracking Search Grid media IDs.

## Related API Operations

- [Declare Media](#)

---

## Select Language Packs

In a system where all media contain the same spoken language and acoustic properties, language pack selection is trivial because only one language pack is likely to have been installed. But if the system contains media with multiple languages and/or multiple acoustic properties, multiple language packs may be required to optimize search quality. In this case, the client application must choose an appropriate language pack for phonetic index creation and subsequent searching.

Individual language packs are identified with a *language pack GUID* available in the language pack's documentation. Language pack GUIDs are also found in each language pack's manifest file, which is available via the Search Grid API.

## Related API Operations

- [Request Media Operation](#)
- [Get Language Packs for Language](#)

- [Get Language Pack Info](#)
- [Get Language Pack Manifest](#)
- [Begin Search](#)

---

## Initiate Phonetic Indexing

Because phonetic indexes are required to support phonetic search, the client application must instruct Search Grid to create an appropriate index for each media file. Once the media is declared, the client application requests that a pat be created for the media with the desired language pack. On a Search Grid instance configured to support LSS, phonetic digest creation proceeds automatically after the pat is created.

In most environments, the client application should request only one pat per media. But in some cases, multiple pats per media may be appropriate. One example is when the audio contains more than one spoken language. Another example is when an updated language pack version is installed. Because searches are constrained to a single language pack, it may make sense to have pats for both the old and new language packs while transitioning the new version.

### Related API Operations

- [Declare Media](#)
- [Request Media Operation](#)

---

## Optional Client Application Functions

---

### Select a Tenant

A tenant is the basic unit of data partitioning, authentication, and authorization in Search Grid. Media files are declared as being associated with a single tenant, and all information related to that media (like phonetic indexes) are associated with that tenant as well. A tenant represents a strict logical data partition: A user presenting credentials for tenant A cannot add, inspect, search, or drop media belonging to tenant B.

If there is no business need to use multiple tenants in Search Grid, the client application can simply use the default tenant that is available when Search Grid is installed. In a multi-tenant environment, the client application must provide the external tenant ID defined when the tenant was created.

Client applications can also perform several aspects of tenant management, such as creating, listing, and deleting tenants. All such operations require administrator credentials. But note that changing passwords requires the use of the Management Console (see Administration Guide), and cannot be achieved via the API.

### Related API Operations

- [Find Tenant](#)
- [Get Tenant Info](#)

---

## Initiate Classification

A client application that can make use of segments produced by classification must instruct Search Grid to classify the media. The most efficient way to do this (in terms of CPU resources) is to request classification and pat creation as part of the same media operation. However, classification may also be requested before or after pats are created.

### Related API Operations

- [Declare Media](#)
- [Request Media Operation](#)

---

## Migrate Pats

In most environments, pats will remain in the Search Grid application for their entire lifecycle, from creation until the pat is deleted or the media is dropped. But if Search Grid is being integrated with other NiCE products, it may be desirable to transfer pats between applications. This saves the expense of creating the same pat multiple times. Note that Search Grid supports only pats in the version 7 and version 8 formats. Consult the documentation for the NiCE product being integrated for details on supported pat versions.

### Related API Operations

- [Add Pat](#)
- [Find Pat](#)
- [Get Pat Inventory](#)

---

## Media Declaration Availability

Search Grid offers two modes that affect the behavior of media declaration: *normal availability* mode and *high availability* mode. Each mode offers distinct advantages and consequences for client application logic. To compare the two modes, consider the following sequence:

1. Client application invokes Declare Media.
2. Search Grid responds with a new media ID.
3. Client application terminates prior to persisting the media ID.

4. Client application restarts.
5. Client application invokes Declare Media for the same media.

Search Grid's response depends on the media declaration availability mode and the status of the Data Nodes:

- In normal availability mode,
  - if all Data Nodes are up,
    - Search Grid returns the same media ID from step 2.
  - else (any Data Node is down),
    - Search Grid rejects the media declaration in step 5 and returns an error.
- In high availability mode,
  - if all Data Nodes are up,
    - Search Grid returns the same media ID from step 2.
  - else if at least one Data Node is up,
    - If one of the up Data Nodes is the one that stored the media info in step 2.
      - Search Grid returns the same media ID from step 2.
    - else (the node that stored the media info in step 2 is down),
      - Search Grid returns a *new* media ID, different from the one in step 2.
  - else (all Data Nodes are down),
    - Search Grid rejects the media declaration in step 5 and returns an error.

So in normal availability mode, Search Grid guarantees that a given external media ID will receive exactly one grid-assigned media ID. But the cost of enforcing this guarantee is reduced availability. All else being equal, a Search Grid deployment with more Data Nodes will be less available for media declaration than a deployment with fewer Data Nodes.

In high availability mode, the client application's responsibilities are more complex. Notice what happens in the case marked with an asterisk above. Search Grid now has two records for the same media, but the client application is aware of only one of them: The client has "leaked" a media declaration into Search Grid. This has several consequences:

- Because the client application is not aware of the leaked media ID, it will never invoke Drop Media for it, and Search Grid resources associated with the media will never be reclaimed. (The resources in question range from a few hundred bytes to several megabytes per media hour, depending on whether the declaration contained a media

operation request.)

- If the declaration in step 1 included a pat creation request, searches using the "All Media for Language Pack" search space will return duplicate results, one for each media ID.
- If the declaration in step 1 included a media operation request with the notification flag set, the client application will eventually receive a notification referencing a media ID of which the client has no record.
- The Find Media operation will return only one of the two media IDs, but which one is unspecified.

Given a decision to use high availability mode, the client application designer has several options. The simplest is to take no special action. Given the multiple failures that must occur to leak media, episodes may be rare enough to be ignored. And if the client exclusively uses Media List search spaces, then duplicate results will not be returned.

Another option is to periodically reconcile the inventories of Search Grid and the client application. The client application could use the Get Media Inventory operation to determine the media IDs known to Search Grid, and then drop media IDs absent from the client applications' records.

If all media declarations include media operation requests, another option is to use notifications to clean up leaked media. When the client application receives a notification for an unknown media ID, the client could react by invoking Drop Media on the unknown media.

Consult your Search Grid administrator to determine the availability mode of a particular Search Grid instance. The default mode is normal availability.

---

## API Overview

---

### API Breaking Changes From Search Grid 3.1

**Search Grid Media IDs:** All APIs that pass Search Grid Media IDs have been modified. As of Search Grid 3.2, Search Grid's Media ID has been widened from a 32-bit integer value to a 64-bit long integer value and can be any positive value up to  $2^{63} - 1$  (that is 9,223,372,036,854,775,807). Client applications **must** accommodate the full range of possible values.

**Media Bit Vectors:** The formerly optional use of Media Bit Vectors (the <mediaBitVector> field) has been eliminated completely as of Search Grid 3.2.

**License Information:** Licensing info (which includes feature info) has been eliminated completely from APIs as of Search Grid 3.2.

**System Mode:** Request System Mode (POST) has been eliminated completely from APIs as of Search Grid 3.2. Get System Mode (GET) has had the errorsOccurred field removed from the response, and the only two possible modes are now Normal and Maintenance.

---

## Design Notes

- The API design largely adopts the style known as "hypertext as the engine of application state" (HATEOAS; Fielding, 2000).
- Internal URI structure is not specified, with two exceptions:
  - URIs do not have query or fragment components unless explicitly noted.
  - URI query components conform to the encoding rules for application/x-www-form-urlencoded URLs (W3C, 1999).
- *Inventory* means a list of related resources of a given type.
- *Info* means an entity describing aspects of a specific resource.
- All external ID comparisons are ordinal and case-sensitive.

---

## Documentation Conventions

- Because URI structure is not part of the public spec, this document uses "dummy" URIs (e.g. "<http://example/media1>") in place of real URIs. (In contrast, URI query parameters *are* part of the public spec for many resource types.)
- Dummy URIs always use the "http" scheme. But if Search Grid is configured to require HTTPS (see [HTTPS Support](#) section), the actual URIs will have the "https" scheme.

---

## XML Entities

- All XML request and response entities are content type application/vnd.nexidia.searchgrid+xml unless otherwise specified.
- All XML entities are in the "<http://ns.nexidia.com/searchgrid>" namespace unless otherwise specified.
- XML entity examples in this document omit namespace declaration and preamble for clarity.
- Any request that contains an XML entity may also contain an x-nx-content-mic header. Search Grid will use the Message Integrity Check value to check the entity's integrity and return a ChecksumMismatch error response if the MIC values do not match.

---

## Error Responses

HTTP error responses include several types of information:

- The *request ID* is a string identifying the client application's HTTP request. It can be used to correlate error events with Search Grid log files.
- The error *code* is a well-known string signifying the type of error. The error code is intended for automated processing, but it is human-readable as well.
- The error *resource* is a URI indicating a resource relevant to the error. In most cases, the error resource is the same as the HTTP request URI. But some errors point to a different resource; for example, to indicate a conflict.
- The error *message* is a human-readable string describing the error. Error messages are intended for administrator or technical support users, so client applications should not present Search Grid error messages directly to end users. Error message content can change from release to release, so client applications should base error handling logic on the error code, not the error message.

Error information is returned within the HTTP headers and, when applicable, in HTTP response entities. The following HTTP headers are available in error responses:

- `x-nx-request-id`: Contains the request ID. This header is available in all Search Grid HTTP responses, but it is most useful in error situations.
- `x-nx-error-code`: Contains the error code.
- `x-nx-error-resource`: Contains the error resource URI.

If the client's HTTP request indicated an Accept header value of "application/vnd.nexidia.searchgrid+xml", error responses include an entity with the structure shown below. (Note that the error entity is not included in responses to HEAD requests.)

```
<error>
  <code>DuplicateExternalMediaId</code>
  <message>Media with the same external media ID already exists.</message>
  <resource href="http://example/mediainventory"/>
  <requestId>7c83aba7-4bc1-4eb4-b0aa-1386df67eaff</requestId>
</error>
```

---

## Global Error Responses

The following error responses are applicable to all API operations. (Message text is not considered part of the specification, and may change in future Search Grid versions.)

| HTTP Response Code             | Search Grid Error Code   | Message                                                                                                           |
|--------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| 500 (Internal Server Error)    | InternalServerError      | Internal error occurred while processing request.                                                                 |
| 400 (Bad Request)              | BadRequest               | Requested URL or input entity was malformed.                                                                      |
| 413 (Request Entity Too Large) | RequestEntityTooLarge    | Input entity is too large for the server to accept.                                                               |
| 400 (Bad Request)              | LengthMismatch           | Specified length did not match that of the received entity.                                                       |
| 400 (Bad Request)              | ChecksumMismatch         | Specified checksum did not match that of the received entity.                                                     |
| 503 (Service Unavailable)      | ServerBusy               | The application is too busy to process the request.                                                               |
| 503 (Service Unavailable)      | ServiceUnavailable       | The server is currently unable to handle the request due to a temporary overloading or maintenance of the server. |
| 503 (Service Unavailable)      | InitializationInProgress | Request cannot be completed because the server is being initialized.                                              |
| 503 (Service Unavailable)      | ShutdownInProgress       | Request cannot be completed because the server is being shut down.                                                |
| 503 (Service Unavailable)      | MaintenanceMode          | Server is currently performing maintenance.                                                                       |



| HTTP Response Code        | Search Grid Error Code  | Message                                                                        |
|---------------------------|-------------------------|--------------------------------------------------------------------------------|
| 503 (Service Unavailable) | ReadOnlyMode            | Server is currently in read-only mode.                                         |
| 503 (Service Unavailable) | MediaOperationsDisabled | Media operations are disabled while the server is set as backup instance role. |
| 403 (Forbidden)           | Forbidden               | User is forbidden to access this resource.                                     |
| 401 (Unauthorized)        | InvalidCredentials      | Invalid username or password.                                                  |
| 401 (Unauthorized)        | AccountLockout          | Account is locked out.                                                         |

---

## Security

Client requests are authenticated using the HTTP Basic scheme as defined in RFC 2617 (Franks et al., 1999). Users fall into two categories, tenant and administrator, with the following characteristics:

- Tenant users have authorization to declare media, add pats, and execute phonetic searches, but only for the specific tenant that user represents.
- Administrator users have authorization to create, delete, and list tenants as well as execute maintenance operations.
- All authenticated users have authorization to access System and Language Pack resources.

For optimum performance, client applications should send credentials preemptively; that is, do not wait to receive a 401 (Unauthorized) challenge to send credentials.

Consult your Search Grid administrator for username and password information.

---

## User Account Lockout

An administrator can configure Search Grid to lock user accounts as a security measure. An account will be locked when a specific number of authentication failures occurs within a specific span of time. (Note that an HTTP request containing no credentials, i.e. no Authorization header, does not contribute to the authentication failure count.) Once an

account is locked, further operations under that account are prohibited until the account is unlocked. A client application attempting to invoke an operation under a locked account will receive a 401 (Unauthorized) response with "AccountLocked" error code.

A locked user account can be unlocked in multiple ways depending on how the administrator has configured the Search Grid instance. The lockout can expire after a specific length of time has passed, or the Management Console can be used to unlock the account. The administrator can configure the lockout period to be indefinite, in which case Management Console is the only way to unlock the account.

---

## HTTPS Support

As discussed above, Search Grid uses the HTTP Basic authentication scheme, which involves transmitting credentials in (base 64 encoded) cleartext. To encrypt credentials and other user data in transit, an administrator can configure Search Grid to use HTTPS for communication with client applications. (Specifically, Search Grid supports HTTP over TLSv1 and SSLv3.) HTTPS also provides authentication options beyond Search Grid's user-level authentication support.

When Search Grid is configured for HTTPS, its use is mandatory for all communication between Search Grid and client applications. Therefore, the System URI, as well as all URIs provided by Search Grid itself, will contain the "https" scheme rather than "http".

Client applications may need to adapt their processing of server certificates depending on the HTTP client framework in use and on how the administrator has configured the Search Grid instance. Client application developers should consider four areas in particular:

- Host name checking. HTTPS clients commonly check the server's host name against the identity presented in the server's certificate. But if Search Grid is configured with a certificate that does not match its host name, client applications may need to disable server identity checks.
- Server authentication via chain of trust. HTTPS clients typically verify that the server's certificate hierarchy includes a mutually trusted certificate authority. Therefore, client applications must be configured to trust a certificate authority that appears in the Search Grid server's certificate.
- Client authentication via chain of trust. Just as HTTPS clients can authenticate servers via a chain of trust, so HTTPS servers can authenticate clients via a chain of trust. If Search Grid is configured for client authentication, the client must present a certificate hierarchy that includes a certificate authority that Search Grid has been configured to trust.
- Self-signed certificates. A self-signed certificate is a special case chain of trust in which the trust anchor is the entity represented by the certificate. Some HTTPS client frameworks require explicit configuration to accept self-signed certificates.

A common situation in non-production contexts is that Search Grid is configured to use its built-in default certificate. The default certificate is self-signed, so most HTTP client frameworks reject it without explicit reconfiguration. In addition, the default certificate is unlikely to match the server's host name, so host name checking will not succeed.

---

## Content Encoding

All Search Grid resources are capable of accepting and returning entities with gzip content coding, which can be beneficial if network bandwidth consumption is a concern. Note that using gzip increases CPU consumption, so it may not be a good choice for all environments. To send a gzip-encoded request, specify "gzip" in the Content-Encoding header. To receive gzip-encoded responses, specify "gzip" in the Accept-Encoding header. For detailed information on the use of content coding in HTTP, see RFC 2616.

Requests that include large search spaces, such as media lists, are particularly good candidates for gzip encoding. However, gzip encoding is not recommended for entities containing binary phonetic artifacts, e.g. the Add Pat and Get Pat Image operations. Phonetic artifacts do not compress well, so gzip coding will consume CPU resources without yielding any benefit.

---

## Notifications

Notifications are a mechanism for a client application to efficiently receive information about multiple asynchronous operations. Notifications are managed through a persistent queuing mechanism. When an event (such as completing a media operation request) triggers a notification, the notification is placed in a persistent queue for the client application to consume.

For example, a client application may need to monitor media operation progress for a batch of new media. This can be achieved by polling the status of each individual media operation request, but if there are 50,000 such requests outstanding, polling each request's status would be very inefficient. With notifications, the client application can poll a single resource to receive notifications about all the tenant's media operation requests as they are completed. See "Create pats for new media" in the [Example Workflows](#) section for pseudocode examples.

Because the notification infrastructure is designed for high reliability, client applications must explicitly delete notifications that they have received. This prevents data loss in the case where a client receives a notification but terminates before it can process the information. As a consequence, notifications may be delivered multiple times in exceptional situations. Consider the following scenario:

1. Client receives notification batch containing notifications A, B, and C.
2. Client processes A and B.
3. Client crashes.
4. Client restarts and resumes notification processing.
5. Client receives notification batch containing notifications A, B, and C.
6. Client processes A, B, and C.
7. Client deletes previous notification batch.

8. Client receives notification batch containing notifications D, E, and F.



In step 5, notifications A, B, and C are delivered to the client application a second time. The client's processing logic in steps 2 and 6 must allow for this possibility. Also note that for efficiency, steps 7 and 8 are combined into a single API operation called "Delete Previous Notification Batch and Get Next".

A single client application instance should retrieve all notifications for a given tenant. If multiple application instances (or multiple threads in a single application instance) attempt to retrieve notifications for the same tenant, they will receive redundant notifications.

Notifications that are not retrieved will eventually expire and be deleted from the queue. Consult your administrator for information about the expiration time.

---

## Media Batches

A *media batch* is a data structure that references a collection of media. Search Grid supports multiple ways of expressing media batches in XML entities, including `mediaList` and `mediaInventory`. Operations may accept multiple forms of media batch in request entities as described in each operation's specification.



As of Search Grid 3.2, the XML entity `mediaBitVector` is no longer supported.

Some Search Grid operations allow the caller to specify what type of media batch structure to return by specifying `media-batch-type` as a MIME subtype parameter in the Accept header like so:

Accept: application/vnd.nexidia.searchgrid+xml; media-batch-type=mediaList

Each operation's specification describes its default media batch type and the alternatives that it supports. See [Other Entities](#) for a description of media batch data structures and syntax.

See RFC 2045 for general information on MIME parameters. See RFC 2616 for more information on the use of MIME parameters in HTTP.

---

## Example Workflows

Examples do not show URI caching although clients are encouraged to cache frequently used URIs such as tenant and search inventory.

---

### Create pats for new media

- Get System Info (provides tenant inventory URI)
- Find Tenant (provides specific tenant URI)

- Get Tenant Info (provides media and notification queue inventory URIs)
- Declare Media (includes pat creation request; provides media operation inventory URI)
- Get Notification Queue Inventory identify notification queue named "mediaOperation" (provides notification queue URI)
- Get Notification Batch

```
do
  sleep until time to poll
  Delete Notification Batch and Get Next
until media operation notification is received
```

---

## Create pats for existing media

- Get System Info (provides tenant inventory URI)
- Find Tenant (provides specific tenant URI)
- Get Tenant Info (provides media and notification queue inventory URIs)
- Find Media (provides specific media URI)
- Get Media Info (provides media operation inventory URI)
- Request Media Operation (provides media operation URI)

### Option A: Notify on pat availability

- Get Notification Queue Inventory
- identify notification queue named "mediaOperation" (provides notification queue URI)
- Get Notification Batch

```
do
  sleep until time to poll
  Delete Notification Batch and Get Next
until media operation notification is received
```

### Option B: Poll for specific pat availability

```
do
  Get Media Operation Info
until media operation is complete
```

---

## Search

- Get System Info (provides tenant inventory URI)
- Find Tenant (provides specific tenant URI)
- Get Tenant Info (provides search inventory URI)
- Begin Search (provides specific search URI)

```
while search is in progress
  Get Search Info (provides result batch URIs)
  for each result batch
    Get Result Batch
    Delete Result Batch
  end for
end while
```

- End Search

---

## Add pre-existing pats for new media

- Get System Info (provides tenant inventory URI)
- Find Tenant (provides specific tenant URI)
- Get Tenant Info (provides media inventory URI)
- Declare Media (provides pat inventory URI)
- Add Pat

---

## Download pat

- Get System Info (provides tenant inventory URI)
- Find Tenant (provides specific tenant URI)
- Get Tenant Info (provides media inventory URI)
- Find Media (provides media URI)
- Get Media Info (provides pat inventory URI)
- Find Pat (provides pat URI)

- Get Pat Image

---

## Drop media in batch

- Get System Info (provides tenant inventory URI)
- Query Tenant (provides specific tenant URI)
- Get Tenant Info (provides media batch drop inventory URI)
- Begin Media Batch Drop (provides specific media batch drop URI)

```
while drop is in progress
  Get Media Batch Drop Info
end while
```

- Get Media Batch Drop Result
- End Media Batch Drop

---

## Tips for Client Application Development

- Use multiple connections and multiple threads. Declaring thousands of media and requesting pat creation for each of them on a single thread is likely to bottleneck the progress of phonetic indexing. Retrieving search result batches is another area where multiple client threads can benefit performance.
- Use persistent HTTP connections, reusing the connections for multiple requests. Reusing a persistent connection is much more efficient than creating a new connection for each request. Also, a client that does not reuse HTTP connections is likely to exhaust its ephemeral TCP ports, especially when declaring large numbers of media.
- Send the correct credentials for each operation. The admin account is not authorized for most tenant-specific operations such as declaring media.
- Send credentials pre-emptively rather than waiting to receive a 401 (Unauthorized) response.
- Be sure to state the correct MIME type in Accept and Content-Type headers. Search Grid operations use a variety of MIME types, and specifying the wrong type for a given operation is a common mistake.
- Pick the most efficient media batch representation. (See [Media Batch Representations](#) for details.)

---

## Resources

The following outline shows the resource structure of the Search Grid API. Most resources have a corresponding inventory resource that is not shown on the outline.

---

## Outline

- System
  - Mode
- Language Pack
- Tenant
  - Notification Queue
  - Notification Batch
  - Media
    - Pat
    - Segment Batch
    - Media Operation
  - Media Batch Drop
    - Media Batch Drop Result
  - Autothreshold
    - Autothreshold Result Batch
  - Search
    - Search Result Batch
  - Phrase Pack
  - Phrase Recognition Output
  - Compilation
  - Structured Query

---

## Operations



---

## Summary

- [Get System Info](#)
- [Get Administration Info](#)
- [Get System Topology](#)
- [Get System Data Metrics](#)
- [Get System Compute Metrics](#)
- [Get System Search Metrics](#)
- [Get System Disaster Recovery Info Metrics](#)
- [Set Administrator User Password](#)
- [Get Media File Extensions](#)
- [Get System Mode](#)
- [Get Node Data Metrics](#)
- [Get Node Compute Metrics](#)
- [Get Node Search Metrics](#)
- [Get Node Disaster Recovery Metrics](#)
- [Add Language Pack](#)
- [Add Legacy Model Pack](#)
- [Get Language Pack Inventory](#)
- [Find Language Pack](#)
- [Get Language Packs for Language](#)
- [Get Language Pack Info](#)
- [Get Language Pack Manifest](#)
- [Validate Search Term](#)
- [Create Tenant](#)

- [Get Tenant Inventory](#)
- [Find Tenant](#)
- [Get Tenant Info](#)
- [Delete Tenant](#)
- [Set Tenant User Password](#)
- [Unlock Tenant User](#)
- [Get Tenant User Lockout Status](#)
- [Get Tenant Data Metrics](#)
- [Get Notification Queue Inventory](#)
- [Get Notification Batch](#)
- [Delete Notification Batch and Get Next](#)
- [Begin Media Batch Drop](#)
- [Get Media Batch Drop Info](#)
- [Get Media Batch Drop Result](#)
- [End Media Batch Drop](#)
- [Declare Media](#)
- [Get Media Inventory](#)
- [Find Media](#)
- [Get Media Info](#)
- [Drop Media](#)
- [Update Segment Batch](#)
- [Get Segment Batch](#)
- [Delete Segment Batch](#)
- [Request Media Operation](#)
- [Get Media Operation Inventory](#)

- [Get Media Operation Info](#)
- [Cancel Media Operation](#)
- [Add Pat](#)
- [Get Pat Inventory](#)
- [Find Pat](#)
- [Get Pat Info](#)
- [Get Pat Image](#)
- [Delete Pat](#)
- [Begin Autothreshold](#)
- [Get Autothreshold Info](#)
- [End Autothreshold](#)
- [Get Autothreshold Result Batch](#)
- [Delete Autothreshold Result Batch](#)
- [Begin Search](#)
- [Get Search Info](#)
- [End Search](#)
- [Get Search Result Batch](#)
- [Delete Search Result Batch](#)
- [Add Phrase Pack](#)
- [Get Phrase Pack Inventory](#)
- [Find Phrase Pack](#)
- [Get Phrase Pack Info](#)
- [Get Phrase Pack Manifest](#)
- [Delete Phrase Pack](#)
- [Get Phrase Recognition Output Inventory](#)

- [Get Phrase Recognition Output](#)
- [Delete Phrase Recognition Output](#)

See [Structured Queries](#) for structured query operations.

---

## Descriptions

HTTP request and response examples elide some components for readability. Commonly omitted headers include Host, Authorization, and x-nx-content-mic. Paths in HTTP requests are usually replaced with a description of the data structure or operation that provides the URI, such as [\[Administration Info URI provided by Get System Info\]](#). Many entities are described in a different section of the document referenced in the example, such as [\[See Metrics Entities section for entity specification\]](#).

---

## Get System Info

This operation provides the starting point for HATEOAS interactions.

The version element is intended to be human-readable, not machine-readable, so client applications should not attempt to parse its contents. Its format may change in future releases.

All authenticated users have authorization to invoke this operation.

### Request

```
GET [System URI must be provided to client application out-of-band] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<systemInfo>
  <administrationInfo href = "http://example/administrationinfo"/>
  <tenantInventory href="http://example/tenantinventory"/>
  <languagePackInventory href="http://example/languagepackinventory"/>
  <mediaFileExtensions href="http://example/mediafileextensions"/>
  <mode href="http://example/mode"/>
  <version>1.2.3.4 (Documentation Build)</version>
</systemInfo>
```

---

## Get Administration Info

Returns resources that provide an assortment of administration functions for the Search Grid installation. For more information about specific sub-resources, see the following sections:

---

[https://wem-dochub.mindtouch.us/Nexidia\\_Analytics\\_and\\_Data\\_Exchange\\_Framework/NXQC\\_12.5/Grid\\_Documentation/NiC...](https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentation/NiC...)

Updated: Thu, 30 Apr 2026 18:17:39 GMT

Powered by  mindtouch™

- [Get System Topology](#)
- [Get System Data Metrics](#)
- [Get System Compute Metrics](#)
- [Get System Search Metrics](#)
- [Get System Disaster Recovery Info Metrics](#)

Only administrators have authorization to invoke this operation.

## Request

```
GET [Administration Info URI provided by Get System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<administrationInfo>
  <systemTopology href="http://example/systemtopology"/>
  <userInfo href="http://example/userinfo"/>
  <dataMetrics href="http://example/datametrics"/>
  <computeMetrics href="http://example/computemetrics"/>
  <searchMetrics href="http://example/searchmetrics"/>
  <disasterRecoveryMetrics href="http://example/drmetrics"/>
</administrationInfo>
```

---

## Get System Topology

Returns information about Search Grid nodes and services participating in the deployment.

Names for Data Nodes are configured as described in the Administration Guide. Because the set of Data Nodes in the system is known through configuration, `dataNode` elements for all Data Nodes are present even if the nodes' services are down. However, if the services are down, the IP address members will not be present.

Names for Control, Compute, and Gateway Nodes are automatically generated. If services in any of these nodes types are down, the corresponding node element is omitted from the system topology.

Only administrators have authorization to invoke this operation.

## Request

```
GET [System Topology URI provided by Get Administration Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<systemTopology>
  <gatewayNode>
    <name>gateway-XXX</name>
    <publicIpAddress>10.1.1.45</publicIpAddress>
    <gridPrivateIpAddress>10.1.1.45</gridPrivateIpAddress>
  </gatewayNode>
  <dataNode>
    <name>Node1</name>
    <publicIpAddress>10.1.1.46</publicIpAddress>
    <gridPrivateIpAddress>10.1.1.46</gridPrivateIpAddress>
    <baseService>
      <dataMetrics href="http://example/node/dataMetrics"/>
      <disasterRecoveryMetrics href="http://example/node/drMetrics"/>
    </baseService>
    <agentService>
      <searchMetrics href="http://example/node/searchMetrics"/>
      <computeMetrics href="http://example/node/computeMetrics"/>
    </agentService>
  </dataNode>
  <dataNode>
    <name>Node2</name>
    <publicIpAddress>10.1.2.46</publicIpAddress>
    <gridPrivateIpAddress>10.1.2.46</gridPrivateIpAddress>
    <baseService>
      <dataMetrics href="http://example/node/dataMetrics"/>
      <disasterRecoveryMetrics href="http://example/node/drMetrics"/>
    </baseService>
    <!-- Agent Service is down, so not shown. -->
  </dataNode>
  <computeNode>
    <name>Compute-XXX</name>
    <gridPrivateIpAddress>10.1.1.46</gridPrivateIpAddress>
    <computeService>
      <computeMetrics href="http://example/node/computeMetrics"/>
    </computeService>
  </computeNode>
  <controlNode>
    <name>Control-XXX</name>
    <gridPrivateIpAddress>10.1.1.46</gridPrivateIpAddress>
  </controlNode>
</systemTopology>
```

---

## Get System Data Metrics

Returns information about data persisted across the entire system. See the [Metrics Entities](#) section for details about the

members returned.

Only administrators have authorization to invoke this operation.

## Request

GET [System Data Metrics URI provided by Get Administration Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
[See Metrics Entities section for entity specification]

---

## Get System Compute Metrics

Returns information about compute-intensive activity across the entire system. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

## Request

GET [System Compute Metrics URI provided by Get Administration Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
[See Metrics Entities section for entity specification]

---

## Get System Search Metrics

Returns information about search activities across the entire system. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

## Request

GET [System Search Metrics URI provided by Get Administration Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Metrics Entities section for entity specification]

---

## Get System Disaster Recovery Info Metrics

Returns information about Disaster Recovery operations across the entire system. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

## Request

GET [Disaster Recovery Info Metrics URI provided by Get Administration Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Entities Metrics section for entity specification]

---

## Set Administrator User Password

Changes the password used for administrator credentials.

Only administrators have authorization to invoke this operation.

## Request

PUT [User Info URI provided by Get Administration Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<user>
  <password>nfnld*</password>
</user>
```

## Response

HTTP/1.1 204 No Content



---

## Get Media File Extensions

Returns a list of the filename extensions for media files that the Search Grid system is able to decode. Because different codecs may be installed on different servers, the list may change over time depending on what servers are running; only extensions supported on all Search Grid servers are reported.

If invoked with admin credentials, the message member will list any extensions available on some servers but not others. The message text is intended for consumption by administrative or support personnel; it should not be displayed to end users.

All authenticated users have authorization to invoke this operation. However, the message may contain more detailed diagnostic information if invoked with admin credentials.

### Request

```
GET [Media Extensions URI provided by Get System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<mediaFileExtensionsInfo>
  <mediaFileExtension>flv</mediaFileExtension>
  <mediaFileExtension>mp3</mediaFileExtension>
  <mediaFileExtension>vox</mediaFileExtension>
  <mediaFileExtension>wav</mediaFileExtension>
  <message>Extension nmf supported by all nodes except datanode5.</message>
</mediaFileExtensionsInfo>
```

---

## Get System Mode

Possible values for mode are "Normal" and "Maintenance".

All authenticated users have authorization to invoke this operation.

### Request

```
GET [Mode URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<modelInfo>
  <mode>Normal</mode>
  <errorsOccurred>false</errorsOccurred>
</modelInfo>
```

---

## Get Node Data Metrics

Returns information about data persisted to a specific Data Node. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

### Request

```
GET [Node Data Metrics URI provided by Get System Topology] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Metrics Entities section for entity specification]
```

---

## Get Node Compute Metrics

Returns information about compute-intensive activity in a specific Data or Compute Node. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

### Request

```
GET [Node Compute Metrics URI provided by Get System Topology] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Metrics Entities section for entity specification]
```

---

## Get Node Search Metrics

Returns information about search activities in a specific Data or Compute Node. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

### Request

GET [Node Search Metrics URI provided by Get System Topology] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
[See Metrics Entities section for entity specification]

---

## Get Node Disaster Recovery Metrics

Returns information about Disaster Recovery activities in a specific Data Node. See the [Metrics Entities](#) section for details about the members returned.

Only administrators have authorization to invoke this operation.

### Request

GET [Node Disaster Recovery Metrics URI provided by Get System Topology] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
[See Metrics Entities section for entity specification]

---

## Add Language Pack

Installs a new language pack to the Search Grid instance. The request entity is an archive file in either zip or gzipped tar (tgz) format containing the files for the language pack. Several rules apply equally to zip and tgz files:

- Language pack version must be compatible; see System Requirements document for a list of compatible versions.
- The archive must contain all files installed with the language pack, including documentation.

- The manifest.xml file should be no more than one directory level below the archive's root.
- Archived files must exhibit the same relative directory structure used when the language pack was installed. Specifically, href attributes in the manifest.xml file must resolve to the correct file locations in the archive.

Uploaded zip files may contain files with the DEFLATED or STORED algorithms. However, phonetic model files do not compress well, so the STORED algorithm is recommended to conserve CPU resources.

Language packs are distributed for Linux in the tgz format. These files can be uploaded directly to Search Grid as-is, without further encoding. Note that the two variations of tgz upload requests differ in their required Content-Type and Content-Encoding header values.



A single language pack may be hundreds of megabytes in size, so this operation may take several minutes to complete. HTTP client frameworks may need higher than normal timeout settings.

Only administrators have authorization to invoke this operation.

### Request (Uploading a Zip File)

```
POST [Language Pack Inventory URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
Content-Type: application/zip
```

[Request entity is the zipped language pack.]

### Request (Uploading a TGZ File)

```
POST [Language Pack Inventory URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
Content-Type: application/x-tar
Content-Encoding: gzip
```

[Request entity is the tarred and gzipped language pack.]

```
POST [Language Pack Inventory URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
Content-Type: application/x-gtar
```

[Request entity is the tarred and gzipped language pack.]

### Response

```
HTTP/1.1 201 Created
Location: [Specific Language Pack URI]
```

[See Get Language Pack Info operation for Language Pack Info entity specification]

```
HTTP/1.1 409 Conflict
x-nx-error-code: ConflictingLanguagePack
```

x-nx-error-resource: [The existing language pack]

---

## Add Legacy Model Pack

Prior to the 8.0 series of language packs, some phonetic models were packaged and distributed separately from the language pack itself. In particular, this strategy was used for search models and phonetic digest models. The Add Legacy Model Pack operation permits installing these legacy models. Syntactically, Add Legacy Model Pack requests are almost identical to those for Add Language Pack, including the supported entity formats. But because the Search Grid API does not expose phonetic models outside a language pack, the response messages are somewhat different.

Successful completion changes the information returned from the Get Language Pack Info operation for the relevant language pack. Installing a search model pack causes the `supportsSearchModel` element to change from false to true. Installing a phonetic digest model pack causes the `supportsLargeScaleSearch` element to change from false to true.

Legacy model packages must include all files installed with the phonetic model, including the manifest.xml file.

Only administrators have authorization to invoke this operation.

### Request (Uploading a Zip File)

POST [Language Pack Inventory URI provided by System Info]?legacyModelPack=true HTTP/1.1  
Content-Type: application/zip

[Request entity is the zipped model pack.]

### Request (Uploading a TGZ File)

POST [Language Pack Inventory URI provided by System Info]?legacyModelPack=true HTTP/1.1  
Content-Type: application/x-tar  
Content-Encoding: gzip

[Request entity is the tarred and gzipped model pack.]

POST [Language Pack Inventory URI provided by System Info]?legacyModelPack=true HTTP/1.1  
Content-Type: application/x-gtar

[Request entity is the tarred and gzipped model pack.]

### Response

HTTP/1.1 204 No Content

HTTP/1.1 403 Forbidden  
x-nx-error-code: ConflictingModelPack  
x-nx-error-resource: [The request URI]

---

## Get Language Pack Inventory

Note that installed version 7.x Search Model Packs and Phonetic Digest Model Packs do not appear in the inventory.

All authenticated users have authorization to invoke this operation.

### Request

```
GET [Language Pack Inventory URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<languagePackInventory>
  <languagePack href="http://example/languagepack1"/>
  <languagePack href="http://example/languagepack2"/>
</languagePackInventory>
```

---

## Find Language Pack

Returns the Language Pack URI of a language pack with the specified language pack GUID.

All authenticated users have authorization to invoke this operation.

### Request

```
GET [Language Pack Inventory URI]?languagePackGuid=<languagePackGuid> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 301 Moved Permanently
Location: [Specific Language Pack URI]

HTTP/1.1 404 Not Found
x-nx-error-code: UnknownLanguagePack
x-nx-error-resource: [The request URI]
```

---

## Get Language Packs for Language

Returns all language packs for the specific language ID. The response entity is a [languagePackInventory](#) containing only installed language packs with the provided language ID. The returned inventory may contain zero, one, or multiple language packs depending on the packs installed in the Search Grid instance.

Use the Get Language Pack Info operation to retrieve the language ID of a specific language pack.

All authenticated users have authorization to invoke this operation.

## Request

```
GET [Language Pack Inventory URI]?languageId=<languageId> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<languagePackInventory>
  <languagePack href="http://example/languagepack1"/>
  <languagePack href="http://example/languagepack2"/>
</languagePackInventory>
```

---

## Get Language Pack Info

Some language packs, especially older ones, do not define values for name, language ID, and/or language name. For such language packs, the name, `languageId`, and/or `languageName` elements (respectively) will be absent.

See the [Add Legacy Model Pack](#) operation for information about the relationship between the `supportsSearchModel` and `supportsLargeScaleSearch` elements and 7.x language packs.

All authenticated users have authorization to invoke this operation.

## Request

```
GET [Language Pack URI provided by Language Pack Inventory or Find Language Pack] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<languagePack>
  <guid>966da303-bde8-4d9d-83e4-d2c49af6ae45</guid>
  <name>NA English Tele 8kHz NMP 7.1.0.59720</name>
  <languageId>0</languageId>
  <languageName>North American English</languageName>
  <supportsSearchModel>true</supportsSearchModel>
  <supportsLargeScaleSearch>true</supportsLargeScaleSearch>
  <manifest href="http://example/languagepackmanifest"/>
  <searchTermValidation href="http://example/searchtermvalidation"/>
```

```
<structuredQueryValidation href="http://example/sqvalidation"/>
</languagePack>
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownLanguagePack
x-nx-error-resource: [The request URI]
```

---

## Get Language Pack Manifest

Note that this operation, unlike most other Grid operations, returns MIME type application/xml.

All authenticated users have authorization to invoke this operation.

### Request

```
GET [Language Pack Manifest URI provided by Language Pack Info] HTTP/1.1
Accept: application/xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/xml

[Entity contains language pack manifest]
```

Note that the XML namespace of the response entity is that of the language pack manifest.

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownLanguagePack
x-nx-error-resource: [The request URI]
```

---

## Validate Search Term

Determines whether a given term is valid for searching.

Validation results include a phoneme count. Often the phoneme count can indicate the efficacy of a search term. In general, terms with more phonemes produce more accurate results. Terms with 3 or fewer phonemes frequently produce inaccurate results.

This operation can optionally attempt to normalize the term (see [Search Term Validation](#)). If the language pack does not support text normalization, validation will proceed on the original term, and no error is returned.

Validation results include a phonetic respelling if the language pack supports respelling, and if the term is respellable (see [Search Term Validation](#)).

All authenticated users have authorization to invoke this operation.



## Request (Without Normalization)

```
GET [Search Term Validation URI provided by Language Pack Info]?queryString=<searchTerm> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Request (With Normalization)

```
GET [Search Term Validation URI provided by Language Pack Info]?queryString=<searchTerm>&
normalize=true HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
200 OK HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
```

Example response entity for a valid search term:

```
<searchTermValidation>
  <queryString>Search for this</queryString>
  <phonemeCount>8</phonemeCount>
  <validForConventionalSearch>true</validForConventionalSearch>
  <validForLargeScaleSearch>true</validForLargeScaleSearch>
</searchTermValidation>
```

Example response entity for a normalized and respelled term.

```
<searchTermValidation>
  <!-- The query string is the normalized term, not the original term. -->
  <queryString>one thousand dollars</queryString>
  <!-- The respelling is of the normalized term, not the original term. -->
  <phoneticRespelling>WON THOW-ZUNNED DAW-LURS</phoneticRespelling>
  <!-- Phoneme count and validity describe the normalized term, not the original term. -->
  <phonemeCount>14</phonemeCount>
  <validForConventionalSearch>true</validForConventionalSearch>
  <validForLargeScaleSearch>true</validForLargeScaleSearch>
</searchTermValidation>
```

Example response entity for a valid search term with a phonetic respelling:

```
<searchTermValidation>
  <queryString>read</queryString>
  <phoneticRespelling>RED</phoneticRespelling>
  <phonemeCount>8</phonemeCount>
  <validForConventionalSearch>true</validForConventionalSearch>
  <validForLargeScaleSearch>true</validForLargeScaleSearch>
</searchTermValidation>
```

Example response entity for a valid standard search term that is not valid for LSS:

```
<searchTermValidation>
```

```

<queryString>the</queryString>
<phonemeCount>2</phonemeCount>
<validForConventionalSearch>true</validForConventionalSearch>
<validForLargeScaleSearch>false</validForLargeScaleSearch>
</searchTermValidation>

```

Example response entity for a valid standard search term against a language pack with no corresponding phonetic digest model:

```

<searchTermValidation>
  <queryString>Search for this</queryString>
  <phonemeCount>8</phonemeCount>
  <validForConventionalSearch>true</validForConventionalSearch>
</searchTermValidation>

```

Example response entity for an invalid search term:

```

<searchTermValidation>
  <queryString>23</queryString>
  <badCharacters>
    <offset>0</offset>
    <offset>1</offset>
  </badCharacters>
  <validForConventionalSearch>false</validForConventionalSearch>
  <validForLargeScaleSearch>false</validForLargeScaleSearch>
</searchTermValidation>

```

- A response of 200 (OK) does not imply that the term is valid for searching, only that validation was executed successfully.
- For languages that use multi-character sequences, multiple characters may be identified as bad even if only one character is in correct.

```

HTTP/1.1 404 Not Found
x-nx-error-code: UnknownLanguagePack
x-nx-error-resource: [The request URI]

```

## Get Tenant Inventory

All authenticated users have authorization to invoke this operation. However, different users receive different content. Administrators receive the full list of tenants. Tenant users receive only a single entry in the inventory, representing that user's tenant.

### Request

```

GET [Tenant Inventory URI provided by System Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml

```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<tenantInventory>
  <tenant href="http://example/tenant1"/>
  <tenant href="http://example/tenant2"/>
</tenantInventory>
```

---

## Create Tenant

The `externalTenantId` must be specified with a minimum length of 1 and maximum length of 64; values may not contain leading or trailing whitespace. The value "admin" cannot be used.

The default password of a new tenant is the same as its external tenant ID. See the Administration Guide for information on how to change the password.

Only administrators have authorization to invoke this operation.

## Request

```
POST [Tenant URI provided by System Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
```

[See Get Tenant Info operation for Tenant Info entity specification]

## Response

```
HTTP/1.1 201 Created
Content-Type: application/vnd.nexidia.searchgrid+xml
Location: [URI to new Tenant Info]
```

[See Get Tenant Info operation for Tenant Info entity specification]

```
HTTP/1.1 400 Bad Request
x-nx-error-code: InvalidExternalId
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 409 Conflict
x-nx-error-code: ConflictingTenant
x-nx-error-resource: [The existing tenant]
```

---

## Find Tenant

Returns the Tenant URI of a tenant with the specified external tenant ID.

All authenticated users have authorization to invoke this operation. However, different users receive different content.

Administrators can query for any tenant. Tenant users can query only their own tenant; Search Grid replies 403 (Forbidden) for any other tenant, even for tenants that do not exist.

## Request

```
GET [Tenant Inventory URI]?externalTenantId=<externalTenantId> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 301 Moved Permanently
Location: [Specific Tenant URI]
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownTenant
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 410 Gone
x-nx-error-code: DeletedTenant
x-nx-error-resource: [The request URI]
```

---

## Get Tenant Info

All authenticated users have authorization to invoke this operation. However, different users receive different content. Administrators can get info for any tenant. Tenant users can get info only for their own tenant; Search Grid replies 403 (Forbidden) for any other tenant, even for tenants that do not exist.

## Request

```
GET [Tenant URI provided by Tenant Inventory or Find Tenant] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<tenant>
  <externalTenantId>Valued Customer</externalTenantId>
  <userInfo href="http://example/userinfo"/>
  <dataMetrics href="http://example/datametrics"/>
  <notificationQueueInventory href="http://example/notificationqueueinventory"/>
  <phrasePackInventory href="http://example/prmodelinventory"/>
  <structuredQueryInventory href="http://example/structuredqueryinventory"/>
  <compilationInventory href="http://example/compilationinventory"/>
  <mediaInventory href="http://example/mediainventory"/>
  <mediaBatchDropInventory href="http://example/mediabatchdrop"/>
  <searchInventory href="http://example/search"/>
  <autothresholdInventory href="http://example/autothreshold"/>
```

```
<creationDateTime>2010-05-30T09:30:10Z</creationDateTime>
</tenant>
```

---

## Delete Tenant

After completion, subsequent calls to the deleted Tenant URI will result in 410 (Gone) or 404 (Not Found) responses. Deleting a tenant also drops all associated media.

Only administrators have authorization to invoke this operation.

### Request

```
DELETE [Tenant URI provided by Tenant Inventory or Find Tenant] HTTP/1.1
```

### Response

```
HTTP/1.1 204 No Content
```

---

## Set Tenant User Password

Changes the password used for tenant credentials.

Administrators and the relevant tenant user have authorization to invoke this operation.

### Request

```
PUT [User Info URI provided by Get Tenant Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<user>
  <password>nfNld*</password>
</user>
```

### Response

```
HTTP/1.1 204 No Content
```

---

## Unlock Tenant User

Unlocks a tenant user that has been locked out. Has no effect if tenant user is not already locked out.

Only administrators have authorization to invoke this operation.

## Request

PUT [User Info URI provided by Get Tenant Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<user>
  <lockedOut>false</lockedOut>
</user>
```

## Response

HTTP/1.1 204 No Content

---

## Get Tenant User Lockout Status

Determines whether a tenant user is currently locked out.

Only administrators have authorization to invoke this operation.

## Request

GET [User Info URI provided by Get Tenant Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<user>
  <lockedOut>false</lockedOut>
</user>
```

---

## Get Notification Queue Inventory

Callers should use the `notificationQueue` element with the desired name attribute. Future Search Grid releases may support additional notification queues with different names.

## Request

GET [Notification Queue Inventory URI supplied by Tenant Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Reply

HTTP/1.1 200 OK

```
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<notificationQueueInventory>
  <notificationQueue href="http://example/notificationqueue1" name="mediaOperation"/>
</notificationQueueInventory>
```

---

## Get Tenant Data Metrics

Returns information about data persisted across the entire system for a specific tenant. See the [Metrics Entities](#) section for details about the members returned.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

```
GET [Tenant Data Metrics URI provided by Get Tenant Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Metrics Entities section for entity specification]
```

---

## Get Notification Batch

Only one actor per tenant should consume notifications. In exceptional conditions (like intermittent network connectivity), the same notification content may be sent multiple times.

The following pseudocode illustrates how a client application should retrieve notifications:

notificationBatch = Get Notification Batch

```
while running
  if notificationBatch contains notifications
    process notifications
  else
    sleep
  end if
  notificationBatch = Delete Notification Batch and Get Next[URI provided by previous notification batch]
end while
```

### Request (Default Notification Count)

```
GET [Notification Queue URI provided by Notification Queue Inventory] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Request (Limited Notification Count)

```
GET [Notification Queue URI]?maxNotificationCount=<maxNotificationCount> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Responses

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<notificationBatch>
  <!-- Any previously specified value of maxNotificationCount is preserved. -->
  <deleteThisAndGetNext href="http://example/q?maxNotificationId=xxx"/>
  <!-- Each notification appears in a separate envelope element. -->
  <!-- The notification id attribute is an opaque string. -->
  <notification id="xxx">
    <notificationDateTime>2010-05-30T09:30:10Z</notificationDateTime>
    <mediaOperation id="f91948cf-3ec8-453a-9430-ff54da90fe86">
      <status>Completed</status>
      <durationMs>18423</durationMs>
      <sampleRateHz>8000</sampleRateHz>
      <media ref="1001"/>
      <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
      <classification>
        <segmentType>5030</segmentType>
        <segmentBatch href="http://example/segmentBatchInventory?segmentType=segmentType1&
segmentType=segmentType2"/>
      </classification>
      <patCreation>
        <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
        <pat href="http://example/pat"/>
      </patCreation>
      <errorOccurred>false</errorOccurred>
    </mediaOperation>
  </notification>
  <notification id="yyy">
    <notificationDateTime>2010-05-30T09:31:10Z</notificationDateTime>
    <mediaOperation id="e33df40f-f03f-4506-bc09-3b5caa1a0f02">
      <status>Completed</status>
      <media ref="1002"/>
      <mediaLocation href="file://mediaserver/mediashare/file2.wav"/>
      <classification>
        <segmentType>5030</segmentType>
      </classification>
      <patCreation>
        <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
      </patCreation>
      <errorOccurred>true</errorOccurred>
      <errorCode>CannotConvertMedia</errorCode>
      <errorMessage>Could not convert the media to a usable format</errorMessage>
    </mediaOperation>
  </notification>
```



```
</notificationBatch>
```

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<notificationBatch>
<!-- The deleteThisAndGetNext URI is provided even if no notifications are available. -->
<deleteThisAndGetNext href="http://example/q/deletethisandgetnext"/>
</notificationBatch>
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownNotificationQueue
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 403 Forbidden
x-nx-error-code: InvalidMaxNotificationCount
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 503 Service Unavailable
x-nx-error-code: ServerBusy
x-nx-error-resource: [The request URI]
```

---

## Delete Notification Batch and Get Next

See [Get Notification Batch](#) operation for more information.

### Request

```
DELETE [Next Notification Batch URI provided by Notification Batch] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

### Responses

See [Get Notification Batch](#) operation for responses.

---

## Declare Media

Client applications should record the value of the `id` attribute of the returned response entity. The `id` value is used to express searches and search results. The `externalMediaId` must be specified with a minimum length of 1 and maximum length of 260; values may not contain leading or trailing whitespace.

As a convenience, the request entity may include segments and media operation requests, eliminating the need to submit separate Update Segment Batch and Request Media Operation after each media declaration. Note that no URI is provided for media operations submitted in this fashion. Client applications that need to ascertain when the media operations are complete should use notifications. See [Request Media Operation](#) for details.

The Media Declaration Availability section describes additional client considerations for using Declare Media.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request (Without Segments or Media Operations)

```
POST [Media Inventory URI provided by Tenant Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

[See Get Media Info for Media Info structure]

## Request (With Segments and Media Operations)

```
POST [Media Inventory URI provided by Tenant Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

```
<media>
  <externalMediaId>urn:media:6e8bc430</externalMediaId>
  <!-- zero or one segmentBatchInventory -->
  <segmentBatchInventory>
    <!-- one or more segmentBatch elements -->
    <segmentBatch>
      <segmentType>4011</segmentType>
      <segment>
        <segmentValue>1</segmentValue>
        <startMs>0</startMs>
        <endMs>34940</endMs>
        <score>50.2</score>
      </segment>
      <segment>
        <segmentValue>0</segmentValue>
        <startMs>34940</startMs>
        <endMs>63200</endMs>
        <score>20.2</score>
      </segment>
    </segmentBatch>
  </segmentBatchInventory>
  <!-- zero or one mediaOperationInventory -->
  <mediaOperationInventory>
    <!-- exactly one mediaOperationRequest element -->
    <mediaOperationRequest>
      <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
      <notifyOnCompletion>true</notifyOnCompletion>
      <classification>
        <!-- one or more segmentType elements -->
        <segmentType>4010</segmentType>
        <segmentType>4600</segmentType>
        <segmentType>5030</segmentType>
      </classification>
      <patCreation>
        <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
      </patCreation>
```

```
</mediaOperationRequest>
</mediaOperationInventory>
</media>
```

## Response

```
HTTP/1.1 201 Created
Content-Type: application/vnd.nexidia.searchgrid+xml
Location: [New media URI]
```

[See Get Media Info operation for Media Info entity specification]

```
HTTP/1.1 400 Bad Request
x-nx-error-code: InvalidExternalId
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 409 Conflict
x-nx-error-code: DuplicateExternalMediaId
x-nx-error-resource: [The existing media]
```

---

## Get Media Inventory

Returns a list of all media for the tenant.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

By default, this operation's response entity contains a [mediaInventory](#) element.

See [Media Batch Representations](#) for more information about media inventory structures.

## Request

```
GET [Media Inventory URI provided by Tenant Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<mediaInventory>
  <media href="http://example/media1"/>
  <media href="http://example/media2"/>
  <media href="http://example/media3"/>
</mediaInventory>
```

## Optional Query Parameters

As of Search Grid 20.0.0, the Get Media Inventory URI accepts optional query parameters that allow querying a subset of the media inventory. If these parameters are omitted, then the full inventory is returned as always.

To specify optional query parameters in the URI, simply append `?paramName1=value1&paramName2=value2`, etc. The optional parameters are:

- `declaredAtOrAfter=<DateTimeString>`
- `declaredBefore=<DateTimeString>`

It is valid to provide none, either, or both of these optional parameters, though if providing both, the `declaredBefore` value must be later than the `declaredAtOrAfter` value.

The value of `<DateTimeString>` value must conform to the following subset of ISO-8601 date/time formats:

```
YYYY-MM-DDThh:mm:ss[.uuu][Z|tzdelta]
```

Where:

- Milliseconds (.uuu above) are allowed but are optional.
- If the time zone information is omitted, the date/time value is presumed to be that of the LOCAL time of the Search Grid gateway.
- If the time zone information is included, it must be either Z (to indicate the time is UTC, aka Zulu time) or an offset in the format of a plus or minus sign, followed by the hh:mm delta from UTC that the time represents.

For example, these all represent the same moment in time:

```
2024-04-19T22:06:39Z
2024-04-19T22:06:39+00:00
2024-04-19T18:06:39
2024-04-19T18:06:39-04:00
2024-04-20T03:36:39+05:30
```

The **first two** being *UTC*, the **third** being *the local time of where the Search Grid gateway is located* (presumed to be Atlanta, GA for the sake of this example), the **fourth** being *Eastern Daylight Time in the U.S.*, and the **fifth** being *India Standard Time*.



In order to eliminate issues that may exist if the time zone of Search Grid's gateway is uncertain, it is recommended to always use one of the formats that specify the time zone delta from UTC that the specified time represents.

The response will be identical to that shown above

---

## Drop Media

After completion, subsequent calls to the dropped Media URI will result in 410 (Gone) or 404 (Not Found) responses. Dropping media also deletes pats associated with that media.

This operation may be invoked when a client application enforces its media retention policy.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

DELETE [Media URI provided by Media Inventory or Find Media] HTTP/1.1

### Response

HTTP/1.1 204 No Content

---

## Begin Media Batch Drop

This operation is a batch form of the Drop Media operation. It is much more efficient than calling Find Media and Drop Media on many individual media.

The `strict` query parameter affects whether unknown or previously dropped media IDs are treated as errors. See the [Media Batch Drop Info](#) section of Other Entities for details. If strict is not specified, it defaults to true.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

See [Media Batch Representations](#) for more information about media list structures.

### Request (As Media List With Strict Error Checking)

POST [Media Batch Drop Inventory URI provided by Tenant Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Accept: application/vnd.nexidia.searchgrid+xml

```
<mediaList>
  <media ref="1001"/>
  <media ref="1002"/>
  <media ref="1003"/>
</mediaList>
```

### Request (As Media List Without Strict Error Checking)

POST [Media Batch Drop Inventory URI provided by Tenant Info]?strict=false HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Accept: application/vnd.nexidia.searchgrid+xml

```
<mediaList>
  <media ref="1001"/>
  <media ref="1002"/>
  <media ref="1003"/>
</mediaList>
```

## Response

HTTP/1.1 201 (Created)  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Location: [Specific Media Batch Drop URI]

[See Other Entities section for Media Batch Drop Info entity specification]

---

## Get Media Batch Drop Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Media Batch Drop URI provided by Begin Media Batch Drop operation] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Other Entities section for Media Batch Drop Info entity specification]

---

## Get Media Batch Drop Result

This operation provides media IDs that were dropped by a media batch drop operation. Under normal circumstances, the returned media batch will be identical to the input media batch passed to Begin Media Batch Drop. But the returned media set may differ from the input media set for the following reasons:

- The input media set included media that were not known to the system, e.g. because they had already been dropped.
- An internal error prevented media batch drop from completing successfully.

This operation's response entity contains a `mediaList`.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

See [Media Batch Representations](#) for more information about media list structures.

## Request

GET [Media Batch Drop Result URI provided by Media Batch Drop Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
<mediaList>  
  <media ref="1001"/>  
  <media ref="1002"/>  
  <media ref="1003"/>  
</mediaList>

---

## End Media Batch Drop

Clients should invoke this operation after the media batch drop is complete and the Media Batch Drop Result (if desired) is retrieved.

After this operation is complete, the specified Media Batch Drop Info resource and its sub-resources are no longer available to clients.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

DELETE [Media Batch Drop URI provided by Begin Media Batch Drop operation] HTTP/1.1

## Response

HTTP/1.1 204 No Content

---

## Find Media

Returns the Media URI of media with the specified media ID or external media ID.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request (By Media ID)

GET [Media Inventory URI]?mediaId=<mediaId> HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Request (By External Media ID)

```
GET [Media Inventory URI]?externalMediaId=<externalMediaId> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 301 Moved Permanently
Location: [Specific Media URI]
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownMedia
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 410 Gone
x-nx-error-code: DroppedMedia
x-nx-error-resource: [The request URI]
```

---

## Get Media Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

```
GET [Media URI provided by Media Inventory or Find Media] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<!-- The id attribute is a 32-bit signed integer unique to this tenant in the Grid instance. -->
<media id="1001">
  <externalMediaId>urn:media:6e8bc430</externalMediaId>
  <declarationDateTime>2010-05-30T09:30:10Z</declarationDateTime>
  <segmentBatchInventory href="http://example/segmentbatchinventory"/>
  <patInventory href="http://example/patinventory"/>
  <mediaOperationInventory href="http://example/mediaoperationioninventory"/>
  <phraseRecognitionOutputInventory href="http://example/phraserecognitionoutputinventory"/>
</media>
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownMedia
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 410 Gone
x-nx-error-code: DroppedMedia
x-nx-error-resource: [The request URI]
```



---

## Update Segment Batch

Associates segment batches with the media for use in search result filtering. The submitted segments will replace any existing segments with the same `segmentType` value.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

```
POST [Segment Batch Inventory URI provided by Media Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<segmentBatchInventory>
  <!-- one or more segmentBatch elements -->
  <segmentBatch>
    <segmentType>4011</segmentType>
    <segment>
      <segmentValue>1</segmentValue>
      <startMs>0</startMs>
      <endMs>34940</endMs>
      <score>50.2</score>
    </segment>
    <segment>
      <segmentValue>0</segmentValue>
      <startMs>34940</startMs>
      <endMs>63200</endMs>
      <score>20.2</score>
    </segment>
  </segmentBatch>
</segmentBatchInventory>
```

### Response

```
HTTP/1.1 204 No Content
```

---

## Get Segment Batch

Returns segment batches associated with the media. Specific segment types may be retrieved by specifying one or more `segmentType` query parameter.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request (All Segment Batches)

```
GET [Segment Batch Inventory URI provided by Media Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

## Request (Specific Segment Batch)

```
GET [Segment Batch Inventory URI provided by Media Info]?segmentType=<segmentType> HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

## Request (Multiple Segment Batches)

```
GET [Segment Batch Inventory URI provided by Media Info]?segmentType=<segmentType1>&
segmentType=<segmentType2> HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<segmentBatchInventory>
  <!-- one or more segmentBatch elements -->
  <segmentBatch>
    <segmentType>4011</segmentType>
    <segment>
      <segmentValue>1</segmentValue>
      <startMs>0</startMs>
      <endMs>34940</endMs>
      <score>50.2</score>
    </segment>
    <segment>
      <segmentValue>0</segmentValue>
      <startMs>34940</startMs>
      <endMs>63200</endMs>
      <score>20.2</score>
    </segment>
  </segmentBatch>
</segmentBatchInventory>
```

---

## Delete Segment Batch

Dissociates the specified segment batches from the media. Specific segment types may be retrieved by specifying one or more `segmentType` query parameters. Deleted segments are no longer available for retrieval or search result filtering. No errors are returned if the media had no segment batches of the specified type(s).

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request (All Segment Batches)

```
DELETE [Segment Batch Inventory URI provided by Media Info] HTTP/1.1
```

## Request (Specific Segment Batch)

DELETE [Segment Batch Inventory URI provided by Media Info]?segmentType=<segmentType> HTTP/1.1

## Request (Multiple Segment Batches)

DELETE [Segment Batch Inventory URI provided by Media Info]?segmentType=<segmentType1>&segmentType=<segmentType2> HTTP/1.1

## Response

HTTP/1.1 204 No Content

---

## Request Media Operation

Note that the media location URI is checked only for syntactic correctness at request time. Successfully creating a request does not imply that the media location URI is accessible.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

POST [Media Operation Inventory URI provided by Media Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Media Operation Entities section for Media Operation Request entity specification]

## Response

HTTP/1.1 201 Created  
Location: [Specific Media Operation URI]

[See Media Operation Entities section for Media Operation entity specification]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnsupportedMediaLocationUri  
x-nx-error-resource: [The supplied media location URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 409 Conflict  
x-nx-error-code: ConflictingPat  
x-nx-error-resource: [The existing pat URI]

HTTP/1.1 409 Conflict  
x-nx-error-code: ConflictingMediaOperation

x-nx-error-resource: [The existing incomplete Media operation request URI]

---

## Get Media Operation Inventory

Returns a list of all media operations for the media. Both incomplete and complete media operations are shown. Completed media operations remain in the inventory until they are automatically removed by the system (see [Get Media Operation Info](#)).

This operation is intended primarily for diagnostic purposes. In general, production applications should use notifications to monitor media operations.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Media Operation Inventory URI provided by Media Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
<mediaOperationInventory>  
  <mediaOperation href="http://example/mediaoperation1"/>  
  <mediaOperation href="http://example/mediaoperation2"/>  
  <mediaOperation href="http://example/mediaoperation3"/>  
</mediaOperationInventory>

---

## Get Media Operation Info

Completed Media Operation Info operations, whether successful or not, will remain available for a limited time. Consult your Search Grid administrator for the specific time limit.

This operation is intended primarily for diagnostic purposes. In general, production applications should use notifications to monitor media operations.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Media Operation URI provided by Request Media Operation or Get Media Operation Inventory operations] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Media Operation Entities section for Media Operation entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownMediaOperationRequest  
x-nx-error-resource: [The request URI]

---

## Cancel Media Operation

This operation returns the media operation status at the time of cancellation. The returned status may show that the media operation was already completed. If the media operation is already completed, it is the client's responsibility to delete any persistent results (e.g. pats or segments) that were created; the completion notification, if requested, will still be sent.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

DELETE [Media Operation URI provided by Request Media Operation operation] HTTP/1.1

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Media Operation Entities section for Media Operation entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownMediaOperationRequest  
x-nx-error-resource: [The request URI]

---

## Add Pat

- Preconditions
  - Media is declared.
  - Pat version is compatible (see below)
  - Language pack is installed with the acoustic model used to create the pat.
- Postconditions

- Pat is queued for monogesting.
- Pat is available for retrieval.
- Pat info is available.
- Segments in the added pat are available via Get Segment Batch.

The `allowUpgrade` query parameter controls whether Search Grid will attempt to upgrade old pat versions to a version usable by Search Grid. If `allowUpgrade` is omitted or set to false, the pat version must be an exact match; version 7 for 7.x language packs and version 8 for 8.x language packs. If `allowUpgrade` is set to true and the pat was created with a 7.x language pack, Search Grid will upgrade the pat to version 7. If Search Grid upgrades the pat, subsequent Get Pat Image operations will return the upgraded pat image rather than the original. Note that upgrading affects only the pat version, it does not make a pat compatible with language packs other than the one with which the pat was created.

The `languagePackGuid` query parameter can be omitted to enable automatic language pack selection; however, explicitly specifying `languagePackGuid` is strongly encouraged. If more than one compatible language pack is installed, the operation will fail with an `AmbiguousLanguagePack` error. A client application that does not know the set of installed language packs in advance cannot anticipate this situation.

If the pat to be added contains segments, they will be added to the media's Segment Batch Inventory. Any previously added segments of the same segment type will be replaced.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request (Disallowing Pat Version Upgrade)

```
POST [Pat Inventory URI provided by Media Info]?languagePackGuid=<languagePackGuid> HTTP/1.1
Content-Type: application/octet-stream
Accept: application/vnd.nexidia.searchgrid+xml

[Entity contains the pat image]
```

### Request (Allowing Pat Version Upgrade)

```
POST [Pat Inventory URI provided by Media Info]?allowUpgrade=true&
languagePackGuid=<languagePackGuid> HTTP/1.1
Content-Type: application/octet-stream
Accept: application/vnd.nexidia.searchgrid+xml

[Entity contains the pat image]
```

### Request (With Automatic Language Pack Selection)

```
POST [Pat Inventory URI provided by Media Info] HTTP/1.1
Content-Type: application/octet-stream
```

Accept: application/vnd.nexidia.searchgrid+xml

[Entity contains the pat image]

## Responses

HTTP/1.1 201 Created  
Accept: application/vnd.nexidia.searchgrid+xml  
Location:http://example/pat1

[See Get Pat Info operation for Pat Info entity specification]

HTTP/1.1 409 Conflict  
x-nx-error-code: ConflictingPat  
x-nx-error-resource: [The existing pat URI]

HTTP/1.1 409 Conflict  
x-nx-error-code: ConflictingMediaOperation  
x-nx-error-resource: [The existing incomplete media operation URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: IncompatiblePatVersion  
x-nx-error-resource: [The request URI]

HTTP/1.1 400 Bad Request  
x-nx-error-code: CouldNotLoadPat  
x-nx-error-resource: [The request URI]

HTTP/1.1 400 Bad Request  
x-nx-error-code: AmbiguousLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: IncompatibleLanguagePack  
x-nx-error-resource: [The request URI]

---

## Get Pat Inventory

Returns a list of all pats for the media.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Pat Inventory URI provided by Media Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml
```

```
<patInventory>
  <pat href="http://example/pat1"/>
  <pat href="http://example/pat2"/>
  <pat href="http://example/pat3"/>
</patInventory>
```

---

## Find Pat

Returns the Pat URI for the pat created for the specified media with the specified language pack. (Note that Pat Inventory is a subresource of Media. Therefore, specifying a Pat Inventory implicitly specifies a Media resource as well.)

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

```
GET [Pat Inventory URI]?languagePackGuid=<languagePackGuid> HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 301 Moved Permanently
Location: [Specific Pat URI]
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownPat
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 410 Gone
x-nx-error-code: DeletedPat
x-nx-error-resource: [The request URI]
```

---

## Get Pat Info

The duration returned describes the duration of media that was indexed, accurate to 1/100th of a second. Therefore, the pat duration may be slightly shorter (by less than ten milliseconds) than the corresponding media file.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

```
GET [Pat URI provided by Pat Inventory or Find Pat] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```



## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<pat>
  <IssSearchable>true</IssSearchable>
  <durationMs>115420</durationMs>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
</pat>
```

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownPat
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 410 Gone
x-nx-error-code: DeletedPat
x-nx-error-resource: [The request URI]
```

---

## Get Pat Image

Returns the stored pat image.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

```
GET [Pat URI provided by Pat Inventory or Find Pat] HTTP/1.1
Accept: application/octet-stream
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream

[Entity is the pat image]
```

---

## Delete Pat

After completion, subsequent calls to the deleted Pat URI will result in 410 (Gone) or 404 (Not Found) responses. Subsequent searches will not return hits for the deleted pat, although searches currently in progress may include such hits.

This operation may be invoked in preparation for replacing an existing pat, e.g. because the pat was create with a non-optimal language pack.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

DELETE [Pat URI provided by Pat Inventory or Find Pat] HTTP/1.1

## Response

HTTP/1.1 204 No Content

---

## Begin Autothreshold

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

POST [Autothreshold Inventory URI provided by Tenant Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Accept: application/vnd.nexidia.searchgrid+xml

[See Autothreshold Entities section for Autothreshold Info entity specification]

## Response

HTTP/1.1 201 Created  
Location: [Specific Autothreshold URI]

[See Autothreshold Entities section for Autothreshold Info entity specification]

HTTP/1.1 400 Bad Request  
x-nx-error-code: InvalidSearchTerm  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: DuplicateSearchTermId  
x-nx-error-resource: [The request URI]

HTTP/1.1 503 Service Unavailable  
x-nx-error-code: SearchBusy  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: TooManySearchTerms  
x-nx-error-resource: [The request URI]

---

## Get Autothreshold Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Autothreshold URI provided by Begin Autothreshold operation] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Autothreshold Entities section for Autothreshold Info entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownAutothreshold  
x-nx-error-resource: [The request URI]

---

## End Autothreshold

Clients should invoke this operation immediately after the final autothreshold result batch is retrieved or the autothreshold operation has ended (e.g. due to an error). The server may reject new autothreshold operations or searches if too many autothreshold operations are underway.

After the End Autothreshold operation is complete, the specified Autothreshold resource is no longer available to clients.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

DELETE [Autothreshold URI provided by Begin Autothreshold operation] HTTP/1.1

### Response

HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownAutothreshold  
x-nx-error-resource: [The request URI]

---

## Get Autothreshold Result Batch

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Autothreshold Result Batch URI provided by Autothreshold Info or Search Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
[See Autothreshold Entities section for entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownSearchResultBatch  
x-nx-error-resource: [The request URI]

---

## Delete Autothreshold Result Batch

Clients should invoke this operation immediately after the autothreshold result batch is successfully retrieved. Note that the server may delay the autothreshold operation if too many result batches are left on the server.

Only the tenant user for the relevant tenant may invoke this operation.

## Request

DELETE [Autothreshold Result Batch URI provided by Autothreshold Info or Search Info] HTTP/1.1

## Response

HTTP/1.1 204 No Content  
  
HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownSearchResultBatch  
x-nx-error-resource: [The request URI]

---

## Add Phrase Pack

Installs a new phrase pack to the Search Grid instance. The request entity is an archive file in either zip or gzipped tar (tgz) format containing the phrase pack files. Several rules apply equally to zip and tgz files:

- The archive must contain all files installed with the phrase pack, including documentation.
- The manifest.xml file should be no more than one directory level below the archive's root.
- Archived files must exhibit the same relative directory structure used when the language pack was installed. Specifically, href attributes in the manifest.xml file must resolve to the correct file locations in the archive.

Uploaded zip files may contain files with the DEFLATED or STORED algorithms. However, phonetic model files do not compress well, so the STORED algorithm is recommended to conserve CPU resources.



A phrase pack can be very large: File sizes above one gigabyte are common. Therefore, expect this operation may take several minutes to complete. HTTP client frameworks may need higher than normal timeout settings.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request (Uploading a Zip File)

```
POST [Phrase Pack Inventory URI provided by Tenant Info] HTTP/1.1
Content-Type: application/zip
Accept: application/vnd.nexidia.searchgrid+xml
```

[Request entity is the zipped phrase pack files.]

## Request (Uploading a TGZ File)

```
POST [Phrase Pack Inventory URI provided by Tenant Info] HTTP/1.1
Content-Type: application/x-tar
Content-Encoding: gzip
Accept: application/vnd.nexidia.searchgrid+xml
```

[Request entity is the tarred and gzipped phrase pack files.]

```
POST [Phrase Pack Inventory URI provided by Tenant Info] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
Content-Type: application/x-gtar
```

[Request entity is the tarred and gzipped phrase pack files.]

## Response

```
HTTP/1.1 201 Created
Content-Type: application/vnd.nexidia.searchgrid+xml
Location: [Specific Phrase Pack URI]
```

```
<phrasePack>
  <guid>e067e28a-cc7b-4061-a8c9-d43ec10ce3fd</guid>
  <name>NAEnglish Call Center Phrase Recognition 1.0</name>
  <languageId>0</languageId>
  <languageName>North American English</languageName>
  <manifest href="http://example/phrasepackmanifest"/>
</phrasePack>
```

```
HTTP/1.1 400 Bad Request
x-nx-error-code: InvalidPhrasePack
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 409 Conflict
```

x-nx-error-code: ConflictingPhrasePack  
x-nx-error-resource: [The existing phrase pack]

---

## Get Phrase Pack Inventory

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Phrase Pack Inventory URI provided by Tenant Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
  
<phrasePackInventory>  
 <phrasePack href="http://example/phrasepack1"/>  
 <phrasePack href="http://example/phrasepack2"/>  
</phrasePackInventory>

---

## Find Phrase Pack

Returns the Phrase Pack URI of a phrase pack with the specified phrase pack GUID.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Phrase Pack Inventory URI]?phrasePackGuid=<phrasePackGuid> HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 301 Moved Permanently  
Location: [Specific Phrase Pack URI]  
  
HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownPhrasePack  
x-nx-error-resource: [The request URI]  
  
HTTP/1.1 410 Gone  
x-nx-error-code: DeletedPhrasePack  
x-nx-error-resource: [The request URI]

---

## Get Phrase Pack Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Phrase Pack URI provided by Phrase Pack Inventory or Find Phrase Pack] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<phrasePack>
  <guid>e067e28a-cc7b-4061-a8c9-d43ec10ce3fd</guid>
  <name>NAEnglish Call Center Phrase Recognition 1.0</name>
  <languageId>0</languageId>
  <languageName>North American English</languageName>
  <manifest href="http://example/phrasepackmanifest"/>
</phrasePack>
```

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownPhrasePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedPhrasePack  
x-nx-error-resource: [The request URI]

---

## Get Phrase Pack Manifest

Note that this operation, unlike most other Grid operations, returns MIME type "application/xml".

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Phrase Pack Manifest URI provided by Phrase Pack Info] HTTP/1.1  
Accept: application/xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/xml

[Entity contains phrase pack manifest]

Note that the XML namespace of the response entity is that of the phrase pack manifest.

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownPhrasePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedPhrasePack  
x-nx-error-resource: [The request URI]

---

## Delete Phrase Pack

After completion, subsequent calls to the deleted Phrase Pack URI will result in 410 (Gone) or 404 (Not Found) responses. Incomplete media operation requests that reference this model will fail with a PhraseRecognitionError code.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

DELETE [Phrase Pack URI provided by Get Phrase Pack Inventory or Find Phrase Pack] HTTP/1.1

### Response

HTTP/1.1 204 No Content

---

## Get Phrase Recognition Output Inventory

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Phrase Recognition Output Inventory URI provided by Media Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<phraseRecognitionOutputInventory>
  <phraseRecognitionOutput href="http://example/phaserecognitionoutput1"/>
  <phraseRecognitionOutput href="http://example/phaserecognitionoutput2"/>
</phraseRecognitionOutputInventory>
```

---

## Get Phrase Recognition Output

Results are organized into three categories:



- *Best word* results represent the phonetic engine's best estimation of the media's speech content in terms of single-word phrases. Ordering the words by their `startMs` values forms a transcript of the media (albeit without correct capitalization or punctuation).
- *Best phrase* results represent the set of multi-word phrases that the phonetic engine recognized with high confidence. Best phrase results may contain interpretations that overlap in time. For example, results may reference the phrases "thank you" and "thank you for calling" at the same time in the file. For this reason, best phrase results cannot be organized into a coherent transcript.
- *Alternate word* represent single-word phrases that the phonetic engine recognized as being possible in the media. Put another way, alternate word results are held to a lower confidence standard than best word results. Alternate word results are like best phrase results in that multiple results may overlap in time.

The same words and phrases tend to occur many times in a given piece of media, so to make the output structure more space-efficient, the string values are factored out into separate `recognizedPhrase` elements. Each `recognizedPhrase` element has a numeric `id` attribute referenced from one or more entries in the result lists. Note in particular that the `recognizedPhrase` elements are *not* listed in the order they were found in the audio.

The `processedSegments` elements describe regions of the media that the phonetic engine analyzed fully. Regions of the media outside the processed segments contain either no speech or speech too unclear for the engine to process.

The example output below is artificially small for readability. Typical phrase recognition output will contain thousands of recognized phrases and tens of thousands of recognized phrase hits. Due to the size of phrase recognition output, it is strongly recommended to call Delete Phrase Recognition Output as soon as the client application completes processing the output.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

```
GET [Phrase Recognition Output URI provided by Phrase Recognition Output Inventory or Phrase Recognition Result] HTTP/1.1
Accept: application/vnd.nexidia.searchgrid+xml
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

<phraseRecognitionOutput>
  <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
  <languageId>1</languageId>
  <recognizedPhrases>
    <recognizedPhrase id="1">HELLO WORLD</recognizedPhrase>
    <recognizedPhrase id="2">WORLD</recognizedPhrase>
    <recognizedPhrase id="3">HELLO</recognizedPhrase>
    <recognizedPhrase id="4">THE LOW</recognizedPhrase>
```

```

    <recognizedPhrase id="5">BURL</recognizedPhrase>
    <recognizedPhrase id="6">WHIRLED</recognizedPhrase>
  </recognizedPhrases>
  <bestWordResults>
    <recognizedPhraseHit>
      <recognizedPhrase ref="3"/>
      <score>85.3951</score>
      <startMs>6400</startMs>
      <endMs>6900</endMs>
    </recognizedPhraseHit>
    <recognizedPhraseHit>
      <recognizedPhrase ref="2"/>
      <score>76.0192</score>
      <startMs>7000</startMs>
      <endMs>7250</endMs>
    </recognizedPhraseHit>
  </bestWordResults>
  <bestPhraseResults>
    <recognizedPhraseHit>
      <recognizedPhrase ref="1"/>
      <score>95.4165</score>
      <startMs>6400</startMs>
      <endMs>6900</endMs>
    </recognizedPhraseHit>
    <recognizedPhraseHit>
      <recognizedPhrase ref="4"/>
      <score>64.7090</score>
      <startMs>6400</startMs>
      <endMs>6900</endMs>
    </recognizedPhraseHit>
  </bestPhraseResults>
  <alternateWordResults>
    <recognizedPhraseHit>
      <recognizedPhrase ref="6"/>
      <score>49.5599</score>
      <startMs>7000</startMs>
      <endMs>7250</endMs>
    </recognizedPhraseHit>
    <recognizedPhraseHit>
      <recognizedPhrase ref="5"/>
      <score>71.8227</score>
      <startMs>7000</startMs>
      <endMs>7250</endMs>
    </recognizedPhraseHit>
  </alternateWordResults>
  <processedSegments>
    <phraseRecognitionSegment>
      <startMs>6000</startMs>
      <endMs>8500</endMs>
    </phraseRecognitionSegment>
  </processedSegments>
</phraseRecognitionOutput>

```

HTTP/1.1 404 Not Found

x-nx-error-code: UnknownPhraseRecognitionOutput  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedPhraseRecognitionOutput  
x-nx-error-resource: [The request URI]

---

## Delete Phrase Recognition Output

Due to the size of phrase recognition output, it is strongly recommended to call Delete Phrase Recognition Output as soon as the client application completes processing the output.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

DELETE [Phrase Recognition Output URI provided by Phrase Recognition Output Inventory or Phrase Recognition Result] HTTP/1.1

### Response

HTTP/1.1 204 No Content

---

## Begin Search

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

POST [Search Inventory URI provided by Tenant Info] HTTP/1.1  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Accept: application/vnd.nexidia.searchgrid+xml

[See Search Entities section for Search Info entity specification]

### Response

HTTP/1.1 201 Created  
Location: [Specific Search URI]

[See Search Entities section for Search Info entity specification]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownPhoneticDigest

x-nx-error-resource: [The request URI]

HTTP/1.1 400 Bad Request

x-nx-error-code: InvalidSearchTerm

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: DuplicateSearchTermId

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: TooManySearchTerms

x-nx-error-resource: [The request URI]

HTTP/1.1 503 Service Unavailable

x-nx-error-code: SearchBusy

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: DuplicateSearchQueryId

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: InvalidSearchQuery

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: UnknownStructuredQuery

x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden

x-nx-error-code: IncompatibleStructuredQuery

x-nx-error-resource: [The request URI]

---

## Get Search Info

Client applications should call this method repeatedly to check for available search results.

If a client does not invoke this operation frequently enough, Search Grid will eventually cancel the search. Should this happen, a subsequent invocation of Get Search Info will return the UnknownSearch error code. Consult your Search Grid administrator to determine the required invocation frequency.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Search URI provided by Begin Search operation] HTTP/1.1

Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Search Entities section for entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownSearch  
x-nx-error-resource: [The request URI]

---

## End Search

Clients should invoke this operation immediately after the final result batch is retrieved, sufficient results have been retrieved, or the search has terminated (e.g. due to an error). The server may reject new searches if too many searches exist.

After the End Search operation is complete, the specified search resource is no longer available to clients. The End Search operation also removes all subresources associated with the search, including search result batches. However, in a normal (non-cancellation) use case, the client application should invoke Delete Search Result Batch as results are retrieved rather than relying on End Search to clean up the search result batches.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

DELETE [Search URI provided by Begin Search operation] HTTP/1.1

## Response

HTTP/1.1 204 No Content

---

## Get Search Result Batch

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Search Result Batch URI provided by Search Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

■ [See Search Entities section for entity specification]

---

## Delete Search Result Batch

Clients should invoke this operation immediately after the result batch is successfully retrieved. Note that the server may delay the search if too many result batches are left on the server.

Only the tenant user for the relevant tenant may invoke this operation.

### Request

■ DELETE [Search Result Batch URI provided by Search Info] HTTP/1.1

### Response

■ HTTP/1.1 204 No Content

---

## Media Batch Representations

Several operations accept or return a media batch to express an aggregation of media. Two types of media batch representations are available: media inventory and media list. Not all operation support all media batch representations; see each operation's description for the subset of media batch representations that the operation supports.

---

## Media Inventory

A media inventory represents the batch as a set of HTTP URIs. It is intended to support HTTP navigation only, and so does not include media ID values.

```
■ <mediaInventory>
  <media href="http://example/media1"/>
  <media href="http://example/media2"/>
  <media href="http://example/media3"/>
</mediaInventory>
```

Usage examples:

- [Get Media Inventory](#)

---

## Media List

A media list contains an explicit list of media ID references. A media list is often the simplest media batch representation to use.

■ <mediaList>

```
<media ref="1001"/>
<media ref="1002"/>
<media ref="1003"/>
</mediaList>
```

Usage examples:

- [Begin Search](#)
- [Begin Autothreshold](#)
- [Begin Media Batch Drop](#)

---

## Media Operation Entities

---

### Media Operation Request

The `mediaOperationRequest` describes one or more operations to perform on a specific piece of media. Additional information for several subelements is available elsewhere in this document:

| Element                             | Programmer's Guide Section                            |
|-------------------------------------|-------------------------------------------------------|
| <code>mediaLocation</code>          | <a href="#">Media Location URIs</a>                   |
| <code>languageIdentification</code> | <a href="#">Languages for Language Identification</a> |
| <code>classification</code>         | <a href="#">Segments and Classifiers</a>              |
| <code>phraseRecognition</code>      | <a href="#">Phrase Recognition</a>                    |

```
<mediaOperationRequest>
  <!-- exactly one mediaLocation -->
  <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
  <!-- optional, defaults to zero -->
  <trackId>1</trackId>
  <!-- optional, omitting means "all channels in track" -->
  <channel>0</channel>
  <!-- optional, defaults to false -->
  <notifyOnCompletion>true</notifyOnCompletion>
  <!-- zero or one languageIdentification element -->
  <languageIdentification>
    <!-- two or more language elements -->
```

```

    <language ref="0"/>
    <language ref="16"/>
  </languageIdentification>
  <!-- zero or one classification element -->
  <classification>
    <!-- one or more segmentType elements -->
    <segmentType>4010</segmentType>
    <segmentType>4600</segmentType>
    <segmentType>5030</segmentType>
    <!-- zero or one classifierSettings element -->
    <classifierSettings>
      <!-- each child element is optional -->
      <minSilenceIntervalMs>500</minSilenceIntervalMs>
      <minSpeechDurationMs>1000</minSpeechDurationMs>
      <flareWidthMs>250</flareWidth>
    </classifierSettings>
    <!-- zero or one overrideChannelId element -->
    <overrideChannelId>1</overrideChannelId>
  </classification>
  <!-- zero or more patCreation element -->
  <patCreation>
    <!-- exactly one languagePack -->
    <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  </patCreation>
  <!-- zero or more phraseRecognition elements -->
  <phraseRecognition>
    <!-- exactly one phrasePack -->
    <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
    <!-- zero or one optimizationTarget element -->
    <!-- omit unless directed otherwise by NiCE technical support -->
    <optimizationTarget>1</optimizationTarget>
  </phraseRecognition>
</mediaOperationRequest>

```

---

## Media Location URIs

Media location URIs must conform to the URI standard as specified in RFC 3986 (Berners-Lee, Fielding, & Masinter, 2005). The "file", "http", and "https" schemes are supported.

---

## File System Media Locations

File URIs should be constructed from Microsoft Windows Uniform Naming Convention (UNC) paths with an explicit server name. The following guidelines illustrate the conversion rules:

Convert paths with backslashes to forward slashes.



| Format     | Example                                                                                          |
|------------|--------------------------------------------------------------------------------------------------|
| UNC        | \\myserver\myshare\mymedia.wav                                                                   |
| URI        | <a href="file:///myserver/myshare/mymedia.wav">file:///myserver/myshare/mymedia.wav</a>          |
| URI in XML | href=" <a href="file:///myserver/myshare/mymedia.wav">file:///myserver/myshare/mymedia.wav</a> " |

Convert reserved URI characters using percent encoding.

| Format     | Example                                                                                                |
|------------|--------------------------------------------------------------------------------------------------------|
| UNC        | \\myserver\myshare\my#media.wav                                                                        |
| URI        | <a href="file:///myserver/myshare/my%23media.wav">file:///myserver/myshare/my%23media.wav</a>          |
| URI in XML | href=" <a href="file:///myserver/myshare/my%23media.wav">file:///myserver/myshare/my%23media.wav</a> " |

Convert spaces using percent encoding.

| Format     | Example                                                                                                |
|------------|--------------------------------------------------------------------------------------------------------|
| UNC        | \\myserver\myshare\my media.wav                                                                        |
| URI        | <a href="file:///myserver/myshare/my%20media.wav">file:///myserver/myshare/my%20media.wav</a>          |
| URI in XML | href=" <a href="file:///myserver/myshare/my%20media.wav">file:///myserver/myshare/my%20media.wav</a> " |

Convert literal percent characters using percent encoding.

| Format | Example                                                                                           |
|--------|---------------------------------------------------------------------------------------------------|
| UNC    | \\myserver\myshare\my%23media.wav                                                                 |
| URI    | <a href="file:///myserver/myshare/my%2523media.wav">file:///myserver/myshare/my%2523media.wav</a> |

| Format     | Example                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| URI in XML | href=" <a href="file:///myserver/myshare/my%2523media.wav">file:///myserver/myshare/my%2523media.wav</a> " |

Convert reserved XML characters using XML encoding.

| Format     | Example                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------|
| UNC        | \\myserver\myshare\my&media.wav                                                                            |
| URI        | <a href="file:///myserver/myshare/my&amp;media.wav">file:///myserver/myshare/my&amp;media.wav</a>          |
| URI in XML | href=" <a href="file:///myserver/myshare/my&amp;media.wav">file:///myserver/myshare/my&amp;media.wav</a> " |

See Risney (2006) for more information about expressing UNC paths as URIs.

For compatibility, Grid accepts UNC path URIs in the format used by the Java framework. Java converts UNC paths by leaving the authority component empty and incorporating the server name and leading double slashes in the path component. As a result, four forward slashes appear after the scheme.

| Format     | Example                                                                                          |
|------------|--------------------------------------------------------------------------------------------------|
| UNC        | \\myserver\myshare\mymedia.wav                                                                   |
| URI        | <a href="file:///myserver/myshare/mymedia.wav">file:///myserver/myshare/mymedia.wav</a>          |
| URI in XML | href=" <a href="file:///myserver/myshare/mymedia.wav">file:///myserver/myshare/mymedia.wav</a> " |

To support single-server installations, Search Grid accepts URIs constructed from local paths without an explicit server name. However, local URIs are discouraged because they do not support media operations in a distributed environment. The following examples illustrate supported forms of local path URIs:

- explicit "localhost" authority
  - <file:///localhost/share/mymedia.wav>
- implied "localhost" authority
  - <file:///folder/mymedia.wav>

- Microsoft Windows drive letters (both forms are equivalent)
  - `file:/c:/folder/mymedia.wav`
  - [file:///c:/folder/mymedia.wav](#)

File URIs with a single leading slash but no drive letter will produce a relative path. Because Search Grid services are unlikely to use the same current directory as the client application, such URIs are discouraged, although they are considered valid:

Discouraged `file:/discouraged/example/mymedia.wav`

---

## File System Media Locations - Further Notes

Construction of invalid file URIs is a common source of integration problems. Most programming frameworks offer functions to convert file paths to URIs. Client application developers should use their framework's routines rather than write conversions from scratch.

Note to .NET users: .NET 4.5 incorrectly handles file paths that contain square bracket characters ("[" and "]"). The framework routines generate an invalid URI with unescaped square brackets, and Search Grid will reject it. The recommended solution is to avoid placing square brackets in file paths. If this is not feasible, then custom URI generation code may be required.

The code snippets below may serve as a guide for the URI encoding step.

Create a URI string given a file path name:

```
// C#
// See above note on paths with square brackets in .NET 4.5.
Uri uri = new Uri(filePathname, UriKind.Absolute);
// Do NOT use Uri.ToString()!
string uriEncodedText = uri.AbsoluteUri;
```

```
// Java
File file = new File(filePathname);
URI uri = file.toURI();
string uriEncodedText = uri.toString();
```

Create a file path name from the URI-encoded string:

```
// C#
Uri uri = new Uri(uriEncodedText, UriKind.Absolute);
string filePathname = uri.LocalPath;
```

```
// Java
URI uri = new URI(uriEncodedText);
File file = new File(uri);
string filePathname = file.getAbsolutePath();
```

---

## Web Server Media Locations

Although this document usually describes Search Grid as a *server* application, Search Grid can also act as a *client*, downloading media from a web server for processing. HTTP-based media access can support integration in environments where accessing media via the file system is not viable.

When accessing media from a web server, Search Grid saves a temporary copy of the downloaded media to disk. Search Grid deletes the temporary file when the media operation is complete. See the Administration Guide for information about securing access to temporary media files.

To use HTTP media access, the client application submits a media operation request with a media location URI that satisfies the following criteria:

- The scheme must be http or https.
- Userinfo content (e.g. username:password) is accepted, but be aware that Search Grid may write the userinfo to log files in cleartext.
  - Example: <http://username:password@server/file.wav>
- The last path element should end in the file extension that you would use when saving the media to a file. Search Grid uses the extension to help determine the media's file format. Submitting a URI without a file extension is legal, but it could interfere with Search Grid's ability to decode the media.
  - Preferred: <http://server/file.wav>
  - Discouraged: <http://server/file>
  - Discouraged: <http://server/file/>
- Port, query, and fragment components are accepted.
  - Example: <http://server:8080/file.wav?site=atl#intro>
- The URI must be reachable from all Search Grid servers.

Search Grid's requests for media have the following characteristics:

- Each request is an HTTP GET operation.
- The Accept header is omitted.
- Search Grid issues requests for media when it is ready to perform the media operation. Depending on number of pending media operation requests, this could be hours or even days after the client application requested the media operation.
- Search Grid may issue many concurrent requests. Consult the Administration Guide for details, but in general, the

peak concurrency level is equal to the total number of logical CPUs on all Search Grid servers.

The following characteristics apply to the web server's responses:

- Search Grid follows 301 and 302 redirection responses. However, Search Grid will inspect only the initial URI for a file extension.
- Response codes in the 3xx, 4xx, or 5xx families (except for specific 3xx codes listed above) will cause Search Grid to complete the media operation with a `CannotAccessMedia` error.
- Specifying a `Content-Length` response header value is optional but strongly encouraged.
- Search Grid ignores `Content-Type` response header values.
- If the response contains an `x-nx-content-mic` header, Search Grid will use it to verify file integrity.

---

## Multi-Channel Media Operations

By default, a media operation applies to all channels in the relevant track, mixed to a single logical channel. It is possible to select a specific channel, both for the operation as a whole and independently for the component operations. This makes it simpler to execute operations on multiple channels of the same file in a single request.

Multi-channel media operations have a few caveats and limitations:

- Only one pat creation component is permitted per language pack. Although pat creation can use the channel override mechanism, this cannot be used to make separate pats for each channel. If multiple channel-specific pats are required, they must be associated with separately declared media.
- Running the same classifier segment types against multiple channels will cause one segment batch to overwrite the other. To prevent this, use `saveAsType` elements to persist the segments with a different type value. For example, a client application could establish segment type values 9010 and 9011 to mean "left channel speech" and "right channel speech". (See [Segments and Classifiers](#) section for information on permissible values to use.)

One common use case for multi-channel media operations is to create a pat covering all channels, but to perform phrase recognition and classification on individual channels. The following request structure can accomplish this:

```
<mediaOperationRequest>
  <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
  <notifyOnCompletion>true</notifyOnCompletion>
  <classification>
    <segmentType>4010</segmentType>
    <saveAsType>9010</saveAsType>
    <overrideChannelId>0</overrideChannelId>
  </classification>
  <classification>
    <segmentType>4010</segmentType>
    <saveAsType>9011</saveAsType>
```

```

    <overrideChannelId>1<overrideChannelId>
  </classification>
  <patCreation>
    <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  </patCreation>
  <phraseRecognition>
    <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
    <overrideChannelId>0<overrideChannelId>
  </phraseRecognition>
  <phraseRecognition>
    <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
    <overrideChannelId>1<overrideChannelId>
  </phraseRecognition>
</mediaOperationRequest>

```

---

## Special Considerations for Phrase Recognition

Phrase recognition is the most computationally intensive operation that Search Grid performs. Media operations that include phrase recognition will typically take two orders of magnitude longer to complete than other types of media operations.

Some media files (roughly between 0.1% and 0.01%) are not compatible with phrase recognition, even if they decode and play back correctly. If Search Grid encounters such a file, the media operation will fail with a `PhraseRecognitionError` code.

Like large scale search, phrase recognition is essentially a statistical operation with a probabilistic component. Therefore, repeating a phrase recognition operation—even with the same media and phrase pack—will not produce identical results. Best word results tend to be the most stable across repeated runs, but best phrase and alternate word results will vary more extensively.

---

## Media Operation Info

Note the `sampleRateHz` element is not available in `mediaOperation` elements that meet the following conditions:

- The media operation was created in Search Grid 1.1 as a pat creation request.
- The media operation completed in Search Grid 1.1.
- The Search Grid instance was upgraded to 1.2.
- The client application retrieves the notification after the upgrade.

```

<!-- Media operation is not yet complete. -->
<mediaOperation id="f91948cf-3ec8-453a-9430-ff54da90fe86">
  <status>InProgress</status>
  <media ref="1001"/>
  <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>

```

```

<languageIdentification>
  <language ref="0"/>
  <language ref="16"/>
</languageIdentification>
<classification>
  <segmentType>4010</segmentType>
  <segmentType>4600</segmentType>
  <segmentType>5030</segmentType>
</classification>
<patCreation>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
</patCreation>
<phraseRecognition>
  <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
</phraseRecognition>
</mediaOperation>
<!-- Media operation completed successfully. -->
<mediaOperation id="f91948cf-3ec8-453a-9430-ff54da90fe86">
  <status>Completed</status>
  <media ref="1001"/>
  <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
  <!-- durationMs and sampleRateHz elements available only if errorOccurred is false -->
  <!-- See note (above) regarding sampleRateHz availability after an upgrade. -->
  <durationMs>18423</durationMs>
  <sampleRateHz>8000</sampleRateHz>
  <languageIdentification>
    <!-- score elements available only if errorOccurred is false -->
    <language ref="0">
      <score>80.4</score>
    </language>
    <language ref="16">
      <score>42.3</score>
    </language>
  </languageIdentification>
  <classification>
    <segmentType>4010</segmentType>
    <segmentType>4600</segmentType>
    <segmentType>5030</segmentType>
    <!-- element available only if errorOccurred is false -->
    <!-- returns only segment types created by this classification -->
    <segmentBatch href="http://example/segmentBatchInventory?segmentType=segmentType1&
segmentType=segmentType2"/>
  </classification>
  <patCreation>
    <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
    <!-- element available only if errorOccurred is false -->
    <pat href="http://example/pat"/>
  </patCreation>
  <phraseRecognition>
    <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
    <phraseRecognitionOutput href="http://example/proutput"/>
  </phraseRecognition>
  <!-- element available only if status is Completed -->
  <errorOccurred>false</errorOccurred>

```

```

</mediaOperation>
<!-- Media operation completed but failed. -->
<mediaOperation id="e33df40f-f03f-4506-bc09-3b5caa1a0f02">
  <status>Completed</status>
  <media ref="1001"/>
  <mediaLocation href="file://mediaserver/mediashare/file1.wav"/>
  <languageIdentification>
    <language ref="0"/>
    <language ref="16"/>
  </languageIdentification>
  <classification>
    <!-- one or more segmentType elements -->
    <segmentType>4010</segmentType>
    <segmentType>4600</segmentType>
    <segmentType>5030</segmentType>
  </classification>
  <patCreation>
    <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  </patCreation>
  <phraseRecognition>
    <phrasePack guidref="e067e28a-cc7b-4061-a8c9-d43ec10ce3fd"/>
  </phraseRecognition>
  <errorOccurred>true</errorOccurred>
  <!-- elements available only if errorOccurred is true -->
  <errorCode>CannotConvertMedia</errorCode>
  <errorMessage>Could not convert the media to a usable format</errorMessage>
</mediaOperation>

```

---

## Media Operation Error Codes

The following list describes possible causes for each media operation error code value. Search Grid application logs, especially from Agent and Compute Service instances, may supply more information.

- **CannotAccessMedia**
  - The file URI is syntactically valid, but does not actually point to the file.
  - The file was deleted.
  - The network or server hosting the file is down.
  - The Agent or Compute Service's user account lacks read permission for the file.
  - The file is open for writing by another application.
- **NoAudioInMedia**
  - The file contains video, but no audio track.



- Note: Search Grid's ability to determine this condition depends on the media file type. Some files may be interpreted as having audio with zero duration rather than having no audio track at all.
- CannotConvertMedia
  - The file uses an encoding that Search Grid does not support.
  - The audio data is encrypted.
  - The required codec is not installed on the Data Node or Compute Node server.
  - The file's sampling frequency is too high or low to support pat creation.
  - The file is corrupt.
  - An IO error caused the file to appear corrupt.
- PhraseRecognitionError
  - Media is not compatible with phrase recognition.
  - Phrase pack was deleted before the operation was complete.
- InvalidTrackId
  - The file does have the requested track `id` number.
- InvalidChannel
  - The file does have the requested channel number.
- InternalError
  - Search Grid lacks free space to store the pat file.
  - Network or IO errors prevent transferring the pat file between nodes.
- AdminCanceled
  - An administrator has canceled the operation. For more information, see the Administratively Canceling Media Operations section of the Administration Guide.

---

## Autothreshold Entities

---

## Autothreshold Request

Every autothreshold request contains two components:

- The autothreshold space defines the set of media to be searched for calculating a threshold.
- The autothreshold expression defines the terms that need a threshold calculated.

---

## Autothreshold Space

An autothreshold request contains exactly one `autothresholdSpace` element. The `autothresholdSpace` element's content defines the set of media to be searched:

- exactly one `languagePack` element
  - Note that language pack GUIDs are available via the Get Language Pack Info operation as well as from the manifest.xml file installed with the language pack.
- at most one of the following media batch representations:
  - `mediaList` element
  - `invariantData` element

---

## All Media for Language Pack

Defines the search space as the set of the tenant's media for which pats are available for the specified language pack.

```
<autothresholdSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
</autothresholdSpace>
```

---

## Media List

A media list contains an explicit list of media IDs to use as input for autothresholding.

See [Media Batch Representations](#) for a detailed description of media lists.

```
<autothresholdSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <mediaList>
    <media ref="1001"/>
    <media ref="1002"/>
    <media ref="1003"/>
  </mediaList>
</autothresholdSpace>
```

```
</mediaList>
</autothresholdSpace>
```

---

## Invariant Data

Autothresholding against invariant data permits the creation of consistent autothreshold values at the expense of accuracy. See the [Autothresholding](#) section for details.

```
<autothresholdSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <invariantData/>
</autothresholdSpace>
```

---

## Autothreshold Expression

Each autothreshold request must contain the following top-level element:

- `autothresholdExpression`

---

## Search Term

Each `searchTerm`'s `id` attribute value must be an integer that is unique within the `autothresholdRequest`. The `id` value is used to correlate result and error information with the search term. Values must be between 1 and  $2^{31} - 1$ , inclusive. Values within an `autothresholdRequest` do not need to be consecutive.

```
<searchTerm id="1">
  <queryString>thank you for calling</queryString>
</searchTerm>
```

Note that the number of search terms that may be specified in one autothreshold request is limited by the Search Grid server. Consult your Search Grid administrator for the specific limit of number of search terms.

---

## Autothreshold Expression

An `autothresholdExpression` element must contain one or more `searchTerm` elements. Note that each contained `searchTerm` element must have an `id` attribute value that is unique within the `autothresholdRequest`. An `autothresholdExpression` element must contain one or more `falseAlarmsPerHour` elements.

```
<autothresholdExpression>
  <searchTerm id="1">
    <queryString>thank you for calling</queryString>
  </searchTerm>
  <searchTerm id="2">
    <queryString>how may i help you</queryString>
```

```
</searchTerm>
<falseAlarmsPerHour>0.10</falseAlarmsPerHour>
<falseAlarmsPerHour>0.05</falseAlarmsPerHour>
</autothresholdExpression>
```

---

## Example Autothreshold Request

POST to autothresholdInventory

```
<autothresholdRequest>
<autothresholdSpace>
  <!-- media batch or all media for language pack -->
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <mediaList>
    <media ref="1001"/>
    <media ref="1002"/>
    <media ref="1003"/>
  </mediaList>
</autothresholdSpace>
<autothresholdExpression>
  <searchTerm id="1">
    <queryString>thank you for calling</queryString>
  </searchTerm>
  <searchTerm id="2">
    <queryString>how may i help you</queryString>
  </searchTerm>
  <falseAlarmsPerHour>0.10</falseAlarmsPerHour>
  <falseAlarmsPerHour>0.05</falseAlarmsPerHour>
</autothresholdExpression>
</autothresholdRequest>
```

---

## Autothreshold Info

```
<autothreshold>
  <status>Completed</status>
  <errorsOccurred>false</errorsOccurred>
  <progress>
    <mediaCountProcessed>64204</mediaCountProcessed>
    <mediaDurationProcessedMs>594903583</mediaDurationProcessedMs>
  </progress>
  <autothresholdResultBatchInventory>
    <autothresholdResultBatch href="http://example/autothresholdresultbatch1"/>
  </autothresholdResultBatchInventory>
</autothreshold>
```

If `errorsOccurred` is true, at least one error occurred during the autothreshold operation, and available results may have been impacted.

"Processed" means "processed without error".

---

## Autothreshold Result Batch

Each `autothresholdResultBatch` element contains one or more `autothresholdResult`.

Each `autothresholdResult` element describes all thresholds for a given search term. An autothreshold operation returns at most one `autothresholdResult` element for a given search term across all autothreshold result batches for that autothreshold operation. Put another way, once a client receives an `autothresholdResult`, the client is guaranteed not to receive another result for that search term in that autothreshold operation.

```
<autothresholdResultBatch>
  <autothresholdResult>
    <!-- exactly one searchTerm per autothresholdResult -->
    <searchTerm ref="1"/>
    <outputThreshold>
      <falseAlarmsPerHour>0.05</falseAlarmsPerHour>
      <threshold>92.94</threshold>
    </outputThreshold>
    <outputThreshold>
      <falseAlarmsPerHour>0.10</falseAlarmsPerHour>
      <threshold>86.34</threshold>
    </outputThreshold>
  </autothresholdResult>
  <autothresholdResult>
    <searchTerm ref="2"/>
    <outputThreshold>
      <falseAlarmsPerHour>0.05</falseAlarmsPerHour>
      <threshold>85.86</threshold>
    </outputThreshold>
    <outputThreshold>
      <falseAlarmsPerHour>0.10</falseAlarmsPerHour>
      <threshold>78.01</threshold>
    </outputThreshold>
  </autothresholdResult>
</autothresholdResultBatch>
```

---

## Search Entities

---

### Search Request

Every search request contains three components:

- The search space defines the set of media to be searched.
- The search expression defines the terms and Boolean operations to search.

- The search options define details of how the search should be executed.

---

## Search Space

A search request contains exactly one `searchSpace` element. The `searchSpace` element's contents defines the set of media to be searched:

- exactly one `languagePack` element
  - Note that language pack GUIDs are available via the Get Language Pack Info operation as well as from the manifest.xml file installed with the language pack.
- at most one `mediaList` element

---

## All Media for Language Pack

Defines the search space as the set of the tenant's media for which pats are available for the specified language pack.

```
<searchSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
</searchSpace>
```

---

## Media List

A media list contains an explicit list of media IDs to search.

See [Media Batch Representations](#) for a detailed description of media lists.

The media list may contain at most one reference to a given media ID. Multiple references to the same media will result in a 400/DuplicateMediaIdInSearchSpace error.

```
<searchSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <mediaList>
    <media ref="1001"/>
    <media ref="1002"/>
    <media ref="1003"/>
  </mediaList>
</searchSpace>
```

---

## Search Expression

Each search request must contain the following top-level element:

- `searchExpression`

---

## Search Term

Each `searchTerm`'s `id` attribute value must be an integer that is unique across `searchTerm` elements in the `searchRequest`. The `id` value is used to correlate result and error information with the search term. Values must be between 1 and  $2^{31} - 1$ , inclusive. Values within a `searchRequest` do not need to be consecutive.

Including more than about twenty search terms will reduce search performance. Attempting to submit more than 32,768 terms in a single search will result in a `TooManySearchTerms` error.

There are several ways to limit the types of hits that Search Grid returns for each search term. The kinds of limitation can be categorized as *quality*, i.e. the precision and accuracy of the hits; and *quantity*, i.e. the number of hits.

Hit quality is controlled by specifying a threshold or a false alarm rate (`falseAlarmsPerHour` element). If a false alarm rate is specified, Search Grid will apply autothresholding to determine the threshold to use. (See the sections on [Confidence Scores](#) and [Autothresholding](#) for more information.) If neither a threshold nor a false alarm rate is specified, Search Grid will autothreshold using a default false alarm rate. Specifying both a threshold and false alarm rate is invalid, resulting in an `InvalidSearchTerm` error.

Hit quantity is controlled by a maximum hit count per media (the `maxHitsPerMedia` element) option. If specified, Search Grid returns no more than the specified number of hits for that term for any one media. Note that the max hit count does not apply across terms (if multiple terms were in the search expression) or across media. If a max hit count is not specified, a default value of 1024 is used. Specifying a value above 1024 is not supported, and will result in an `InvalidSearchTerm` error.

Quality and quantity limits can be combined in multiple ways as described in the following list. (Recall that a quality limit is either a threshold or a false alarm rate.)

- Unspecified quality limit, unspecified quantity limit: The default autothreshold is applied as well as the default max hits per media.
- Explicit quality limit, unspecified quantity limit: The supplied quality limit is applied as well as the default max hits per media.
- Unspecified quality limit, explicit quantity limit: No quality limit is applied, but the max hits per media limit is applied. When using this combination, every file is likely (but not guaranteed) to produce a hit, although many of those hits are likely to be false alarms. This combination is useful only for client applications performing advanced post-processing on confidence scores.
- Explicit quality limit, explicit quantity limit: The supplied quality limit and the supplied max hits per media are both applied.

❏ `<searchTerm id="1">`

```

<queryString>thank you for calling</queryString>
<!-- quality limited by threshold; default quantity limit -->
<threshold>70.0</threshold>
</searchTerm>
<searchTerm id="3">
<queryString>how may I help you</queryString>
<!-- quality limited by false alarm rate; default quantity limit -->
<falseAlarmsPerHour>0.05</falseAlarmsPerHour>
</searchTerm>
<searchTerm id="4">
<queryString>may I have your address</queryString>
<!-- quantity limit only -->
<maxHitsPerMedia>16</maxHitsPerMedia>
</searchTerm>
<searchTerm id="5">
<queryString>may I have your phone number</queryString>
<!-- explicit quality and quality limits -->
<threshold>85.0</threshold>
<maxHitsPerMedia>2</maxHitsPerMedia>
</searchTerm>
<searchTerm id="6">
<queryString>may I have your phone number</queryString>
<!-- explicit quality and quality limits -->
<falseAlarmsPerHour>0.05</falseAlarmsPerHour>
<maxHitsPerMedia>1</maxHitsPerMedia>
</searchTerm>

```

---

## Search Expression

A [searchExpression](#) element must contain one or more elements representing a search predicate. A search predicate can be a term or a structured query; represented as a [searchTerm](#) or [searchQuery](#) element, respectively. (This section describes the use only of [searchTerm](#) elements; see [Structured Queries](#) for information on [searchQuery](#) elements.)

All search predicate elements must contain an [id](#) attribute value that is unique within its predicate type in the [searchRequest](#). So each [searchTerm](#) element must have an [id](#) value that no other [searchTerm](#) has, and so on.

A [searchExpression](#) can have an operator element with a value of all or any. All requires that the media file must contain all search terms listed in the [searchExpression](#). The any operator requires that the media file must contain at least one of the search terms listed in the [searchExpression](#). Operator defaults to any if not specified.

A [searchExpression](#) can have a [timeOperator](#) element with a value of anywhere, first, notFirst, last, notLast, and within. This controls where in the media file to search for the [searchTerms](#). A [timeOperator](#) within requires all the search terms to be found within a time frame of each other. A [timeOperator](#) within requires the operator to be all. The [timeOperator](#) value defaults to anywhere if not specified.





The number of search terms that may be specified in one search request is limited by the Search Grid server. Consult your Search Grid administrator for the specific limit of number of search terms.

```
<searchExpression>
  <searchTerm id="1">
    <queryString>thank you for calling</queryString>
    <threshold>70.0</threshold>
  </searchTerm>
  <searchTerm id="2">
    <queryString>how may i help you</queryString>
    <threshold>85.0</threshold>
  </searchTerm>
  <operator>any</operator>
  <timeOperator>first</timeOperator>
  <timeBasisMs>60000</timeBasisMs>
</searchExpression>
```

---

## Search Options

Search option defaults are defined in the specification and are thus not configurable by administrators. This allows search requests to define a subset of the available options yet produce predictable results on any search grid instance.

Only one `searchOptions` element is permitted within each `searchRequest` element.

---

## Search Algorithm

Values:

- "conventional"
- "largeScale"

```
| <searchAlgorithm>conventional</searchAlgorithm>
```

---

## Priority

The priority option influences resource allocation between multiple concurrent operations. Three values, "high", "normal", and "low", are supported. Searches with a high priority receive more resources (and thus complete more quickly) than searches with normal or low priorities. Similarly, searches with a low priority receive fewer resources (and thus complete more slowly) than searches with high or normal priorities.

A key use of the priority option is to allow interactive searches (i.e. initiated by a human user currently awaiting the results) to take priority over non-interactive searches (i.e. initiated by automated processing). This could be accomplished by the client application specifying a high priority for interactive searches and a low priority for the non-interactive searches.

| <priority>normal</priority>

The priority setting defaults to "normal" if a value is not specified.

---

## Return Intersecting Segments

If `returnIntersectingSegments` is set to true, each hit will be accompanied by all audio segments associated with that media that overlap the hit. For example, given a hit that starts at 6700ms and ends at 10000ms, the following segments overlap:

- 6000ms - 6700ms; segment ends on hit start
- 6000ms - 6800ms; segment end overlaps hit start
- 6000ms - 11000ms; segment contains hit
- 6800ms - 9000ms; segment contained by hit
- 9000ms - 11000ms; segment start overlaps hit end
- 10000ms - 11000ms; segment starts on hit end

The following segments do not intersect with the same hit, and thus would not be returned within the results:

- 6000ms - 6600ms; segment before hit
- 11000ms - 12000ms; segment after hit

Returning intersecting segments may decrease search performance depending on the volume of audio segments associated with the media.

The `returnIntersectingSegments` setting defaults to false if a value is not specified.

---

## Return Result Details

This option affects the information available in returned search results. If `returnResultDetails` is set to false, only "positive" results are returned; that is, results for media that had one or more hits. If `returnResultDetails` is set to true, "negative" results are returned as well, explicitly identifying media with no hits. Setting `returnResultDetails` to true also returns media-specific error information.

Because setting `returnResultDetails` to true can dramatically increase the volume of information returned to the client, applications should avoid its use unless negative results are critical to the client application's behavior. One example would be an application required to display media without hits to the user as quickly as possible. With `returnResultDetails` set to false, the application could not tell which media had no hits until the entire search was complete.

The `returnResultDetails` setting defaults to false if a value is not specified.

```
<returnResultDetails>true</returnResultDetails>
```

---

## Search Model

Boolean, defaults to true.

If true, the system selects and applies an appropriate search model if one is available. The user is not notified if no applicable search model is installed.

```
<applySearchModel>true</applySearchModel>
```

---

## Candidates Per Media Hour

Float, defaults to 1.0. Must be greater than zero. Applies only if `searchAlgorithm` is "largeScale".

```
<candidatesPerMediaHour>0.1</candidatesPerMediaHour>
```

---

## Word-Based Rescoring

Boolean, defaults to false.

If true, searches for multi-word query strings will be less likely to hit utterances of a subset of the words in the string. For example, consider searching for the query string "Fulton County Fire Department" in media containing the utterance "Cobb County Fire Department". With word-based scoring set to false, the search is likely to produce a hit because the two phrases are phonetically similar. But with word-based rescoring set to true, the search is less likely to produce a hit because the word "Fulton" is absent in the media; or if a hit is produced, it will have a lower score than it would have with word-based rescoring set to false.

```
<applyWordBasedRescoring>false</applyWordBasedRescoring>
```

---

## Segment Expression

List of segment constraints. If a hit is rejected by any constraint in the list or not accepted by any constraint it will be rejected

### Segment Constraint

| Element | Valid Values  | Meaning                             |
|---------|---------------|-------------------------------------|
| onMatch | accept reject | What to do if the hit is within the |

| Element      | Valid Values                               | Meaning                                                                                                           |
|--------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
|              |                                            | segment                                                                                                           |
| onNoSegment  | accept reject                              | What to do if there are no segments in the file associated with the segment type                                  |
| matchRule    | hitAllWithinSegment hitStartsWithinSegment | Defaults to hitStartsWithinSegment                                                                                |
| segmentType  | integer                                    | ID of the segment type that created the segment; see <a href="#">Segments and Classifiers</a> section for details |
| segmentValue | integer                                    | The value of the region represented by the segment                                                                |

## Example Search Request

```

<searchRequest>
  <searchSpace>
    <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
    <mediaList>
      <media ref="1001"/>
      <media ref="1002"/>
      <media ref="1003"/>
    </mediaList>
  </searchSpace>
  <searchExpression>
    <searchTerm id="1">
      <queryString>thank you for calling</queryString>
      <threshold>70.0</threshold>
    </searchTerm>
    <searchTerm id="2">
      <queryString>how may i help you</queryString>
      <threshold>85.0</threshold>
    </searchTerm>
    <operator>any</operator>
    <timeOperator>first</timeOperator>
    <timeBasisMs>60000</timeBasisMs>
  </searchExpression>
</searchRequest>

```

```

</searchExpression>
<searchOptions>
  <searchAlgorithm>conventional</searchAlgorithm>
  <priority>high</priority>
  <applySearchModel>true</applySearchModel>
  <segmentExpression>
    <segmentConstraint>
      <onMatch>reject</onMatch>
      <onNoSegments>accept</onNoSegments>
      <matchRule>hitAllWithinSegment</matchRule>
      <segmentType>4010</segmentType>
      <segmentValue>0</segmentValue>
    </segmentConstraint>
    <segmentConstraint>
      <onMatch>accept</onMatch>
      <onNoSegments>reject</onNoSegments>
      <segmentType>6010</segmentType>
      <segmentValue>1</segmentValue>
    </segmentConstraint>
    <segmentConstraint>
      <onMatch>reject</onMatch>
      <onNoSegments>reject</onNoSegments>
      <segmentType>6011</segmentType>
      <segmentValue>1</segmentValue>
    </segmentConstraint>
  </segmentExpression>
</searchOptions>
</searchRequest>

```

---

## Search Info

```

<search>
  <status>InProgress</status>
  <errorsOccurred>false</errorsOccurred>
  <progress>
    <mediaCountProcessed>64204</mediaCountProcessed>
    <mediaCountWithHits>64204</mediaCountWithHits>
    <mediaDurationProcessedMs>594903583</mediaDurationProcessedMs>
    <estimatedTimeRemainingMs>493</estimatedTimeRemainingMs>
  </progress>
  <searchResultBatchInventory>
    <searchResultBatch href="http://example/searchresultbatch1"/>
    <searchResultBatch href="http://example/searchresultbatch2"/>
    <searchResultBatch href="http://example/searchresultbatch3"/>
  </searchResultBatchInventory>
  <autothresholdResultBatchInventory>
    <autothresholdResultBatch href="http://example/autothresholdresultbatch1"/>
  </autothresholdResultBatchInventory>
</search>

```

When the search is complete and all result batches are available for retrieval, the status element will contain the value

"Completed".

If `errorsOccurred` is true, at least one error occurred during the search, and available results may have been impacted.

"Processed" means "processed without error".

The `estimatedTimeRemainingMs` element is optional, and will not appear until a stable time has been calculated.

The `autothresholdResultBatchInventory` will be provided only if thresholds were not provided in the initial search request.

---

## Search Result Batch

Each `searchResultBatch` element comprises one or more `searchResult` elements.

Each `searchResult` element describes all hits for the associated media. A search returns at most one `searchResult` element for a given media across all search result batches for that search. Put another way, once a client receives a `searchResult`, the client is guaranteed not to receive another result for that media in that search. If the `returnResultDetails` option was set to false (the default value), only those search results with hits are returned.

```
<searchResultBatch>
  <searchResult>
    <media ref="1001"/>
    <hit>
      <searchTerm ref="1"/>
      <score>89.3934</score>
      <startMs>34840</startMs>
      <endMs>34940</endMs>
    </hit>
    <hit>
      <searchTerm ref="2"/>
      <score>95.9473</score>
      <startMs>54840</startMs>
      <endMs>54940</endMs>
    </hit>
  </searchResult>
  <searchResult>
    <media ref="1002"/>
    <hit>
      <searchTerm ref="2"/>
      <score>90.9347</score>
      <startMs>54840</startMs>
      <endMs>54940</endMs>
    </hit>
  </searchResult>
</searchResultBatch>
```

If the search was requested with the `returnResultDetails` option set to true, the `searchResultBatch` will contain additional information. Some `searchResults` may contain no hit elements, meaning that the media was searched but contained no hits. Also, every `searchResult` element will contain an `errorOccurred` flag. If `errorsOccurred` is set to true, that media may contain additional hits that could not be discovered due to an internal error. It is possible (though unlikely) for a `searchResult` to contain both hits and a true `errorOccurred` flag.

Example `searchResultBatch` with `returnResultDetails` set to true:

```
<searchResultBatch>
  <searchResult>
    <media ref="1001"/>
    <!-- errorOccurred available in every searchResult -->
    <errorOccurred>false</errorOccurred>
    <hit>
      <searchTerm ref="1"/>
      <score>89.3934</score>
      <startMs>34840</startMs>
      <endMs>34940</endMs>
    </hit>
    <hit>
      <searchTerm ref="2"/>
      <score>95.9473</score>
      <startMs>54840</startMs>
      <endMs>54940</endMs>
    </hit>
  </searchResult>
  <searchResult>
    <media ref="1002"/>
    <errorOccurred>false</errorOccurred>
    <hit>
      <searchTerm ref="2"/>
      <score>90.9347</score>
      <startMs>54840</startMs>
      <endMs>54940</endMs>
    </hit>
  </searchResult>
  <searchResult>
    <!-- searchResult for media with no hits -->
    <media ref="1003"/>
    <errorOccurred>false</errorOccurred>
  </searchResult>
  <searchResult>
    <!-- searchResult for media with no hits and an error -->
    <media ref="1003"/>
    <errorOccurred>true</errorOccurred>
  </searchResult>
</searchResultBatch>
```

If the search was specified with `returnIntersectingSegments` set to true, each hit will be accompanied by any segments that overlap the hit:

```

<searchResultBatch>
<searchResult>
  <media ref="1001"/>
  <hit>
    <searchTerm ref="1"/>
    <score>89.3934</score>
    <startMs>34840</startMs>
    <endMs>34940</endMs>
    <segmentBatchInventory>
      <segmentBatch>
        <segmentType>4011</segmentType>
        <segment>
          <segmentValue>1</segmentValue>
          <startMs>0</startMs>
          <endMs>34940</endMs>
          <score>50.2</score>
        </segment>
        <segment>
          <segmentValue>0</segmentValue>
          <startMs>34940</startMs>
          <endMs>63200</endMs>
          <score>20.2</score>
        </segment>
      </segmentBatch>
    </segmentBatchInventory>
  </hit>
  <hit>
    <searchTerm ref="2"/>
    <score>95.9473</score>
    <startMs>54840</startMs>
    <endMs>54940</endMs>
    <segmentBatchInventory>
      <segmentBatch>
        <segmentType>4011</segmentType>
        <!-- This segment intersected with the previous hit
          for the same media as well, so it appears twice. -->
        <segment>
          <segmentValue>0</segmentValue>
          <startMs>34940</startMs>
          <endMs>63200</endMs>
          <score>20.2</score>
        </segment>
      </segmentBatch>
    </segmentBatchInventory>
  </hit>
</searchResult>
<searchResult>
  <media ref="1002"/>
  <hit>
    <searchTerm ref="2"/>
    <score>90.9347</score>
    <startMs>54840</startMs>
    <endMs>54940</endMs>
    <!-- This hit has no overlapping segments. -->
  </hit>
</searchResult>

```



```
<segmentBatchInventory/>
</hit>
</searchResult>
</searchResultBatch>
```

---

## Metrics Entities

---

### Common Patterns and Members

---

#### Scope

Metrics retrieval operations can be invoked at the System, Node, or Tenant scopes, depending on the resource used to provide the URI. The scope defines the type of aggregation used to calculate the metric values.

- System scope aggregates across all nodes and all tenants in the system. Only administrators have access to system scope metrics.
- Node scope aggregates across all tenants on a single node. Only administrators have access to node scope metrics.
- Tenant scope aggregates across all nodes for a single tenant. Only the tenant user for the relevant tenant has access to tenant scope metrics.

---

#### Consistency

All metric values are eventually consistent. In an active system where media is being declared, media operations are being completed, and so on, it is possible that numbers at different scopes may not "add up". For example, the sum of pending media operations across all Data Nodes may not quite equal the count at the system scope. But in a quiescent system in which no data is being mutated, all metrics will "catch up" to their true values, usually within a few seconds.

---

#### Errors

All metrics entities have an `errorsOccurred` member. If this is true, then Search Grid encountered a problem trying to collect the metrics data. In this situation, the returned metrics may be incomplete, and should be interpreted with caution. Depending on the application, it may not be appropriate to display such incomplete metrics to an end user. The reason partial metrics are returned (versus a standard error response) is that it may be useful to have data about the portion of the system operating normally. For example, if a Data Node is down, data metrics will show `errorsOccurred` set to true, and include metrics about other Data Nodes that were able to respond.

---

## Times Real-Time (xRT)

Several search and compute metrics are described in terms of the unit *times real-time*, abbreviated *xRT*. (In a programmatic context, Search Grid API naming standards result in the capitalization *Xrt*.) This is a ratio of the media duration processed to the wall-clock time required to complete processing. For example, if it takes one minute to create a pat from a one-hour media file, the operation completes at a rate of 60xRT.

$$\begin{array}{l} \text{1 hour of media} = 60 \text{ minutes of media} \\ \frac{60 \text{ minutes of media}}{1 \text{ minute of wall-clock time}} = 60\text{xRT} \end{array}$$

For another example, if it takes two seconds to search across media with a total duration of 15 hours, the search rate is 27,000xRT.

$$\begin{array}{l} \text{15 hours of media} = 54,000 \text{ seconds of media} \\ \frac{54,000 \text{ seconds of media}}{2 \text{ seconds of wall-clock time}} = 27,000\text{xRT} \end{array}$$

---

## Rate Metrics

Rate metrics show information averaged over one, five, and fifteen minute windows. (Strictly speaking, these are *exponentially weighted moving averages*. The averaging algorithm is similar to that used by the *uptime* and *top* commands to describe CPU load on Unix-like systems.) For example, consider the media operation rate members in system compute metrics structures. These describe rate at which media operation are being completed across the system. In the following example snippet, on average over the past minute, the system has completed 26.4 media operations per second. Similarly, on average over the past five and fifteen minutes, the system has completed 14.3 and 8.4 media operations per second, respectively.

```
<mediaOperationRatePerSec01Min>26.4</mediaOperationRatePerSec01Min>
<mediaOperationRatePerSec05Min>14.3</mediaOperationRatePerSec05Min>
<mediaOperationRatePerSec15Min>8.4</mediaOperationRatePerSec15Min>
```

Structuring the information this way lets us get information about the rate of change in the system at a glance. Notice that in this example, the values increase as the window size decreases. This tells us that the number of media operations underway has increased over past fifteen minutes; perhaps a client application is making new media operation requests.

It is important to realize rates describe *throughput* rather than *capacity*. Put another way, the rates describe operations the system actually performed, regardless of any idle resources the system has.

Consider a system observed to have a conventional search rate of 27,000xRT in the morning, but 16,500xRT in the afternoon. Without looking at other metrics, there are two possible explanations: First, competing load on the system

caused search performance to be reduced; second, client applications submitted fewer (or smaller) search requests in the afternoon. In the latter scenario, the system had idle resources during the measurement period, and could have exhibited a higher rate if the client had submitted more (or larger) search requests. In this example, comparing the conventional search load metrics in the morning and the afternoon could help understand the cause.

---

## Data Metrics

All member names that end in *Count* or *Ms* are 64-bit signed integers. All other numeric members are signed 64-bit floating point.

- `mediaCount`
  - The number of many media are declared within the scope.
- `mediaOperationsPendingCount`
  - The number of media operations queued within the scope.
- `languagePackMetrics`
  - Segregates metrics for artifacts associated with a specific language pack.
- `patCount` and `patDurationMs`
  - The number of pats accessible to conventional search for the associated language pack.
  - Due to the eventually consistent nature of metrics, these values may not equal the sum of the pre-LSS and LSS pat metrics.
- `preLssPatCount` and `preLssPatDurationMs`
  - The number of pats accessible to conventional search, but not to large scale search for the associated language pack.
- `lssPatCount` and `lssPatDurationMs`
  - The number of pats accessible to both conventional search and large scale search for the associated language pack.

Example of response data:

```
<dataMetrics>
  <errorsOccurred>false</errorsOccurred>
  <mediaCount>203048</mediaCount>
  <mediaOperationsPendingCount>5049</mediaOperationsPendingCount>
  <!-- Zero or more -->
  <languagePackMetrics>
```

```

<languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
<patCount>203032</patCount>
<patDurationMs>121819200000</patDurationMs>
<preLssPatCount>2</preLssPatCount>
<preLssPatDurationMs>1218192</preLssPatDurationMs>
<lssPatCount>203030</lssPatCount>
<lssPatDurationMs>121819200000</lssPatDurationMs>
</languagePackMetrics>
<languagePackMetrics>
  <languagePack guidref="1dc5621c-6219-4485-acae-d53210b9eefb"/>
  <patCount>98745</patCount>
  <patDurationMs>45968405698406</patDurationMs>
  <preLssPatCount>10</preLssPatCount>
  <preLssPatDurationMs>3450980349</preLssPatDurationMs>
  <lssPatCount>9234098</lssPatCount>
  <lssPatDurationMs>3458973459873</lssPatDurationMs>
</languagePackMetrics>
</dataMetrics>

```

## Compute Metrics

- `mediaOperationLoad`
  - The number of media operations underway in the scope.
- `mediaOperationRateXrt`
  - The throughput of media operations in terms of xRT.
- `mediaOperationRatePerSec`
  - The throughput of media operations in terms of completed operations per second.
- `monogestCreationLoad`
  - The number of monogesting operations underway in the scope.
- `monogestCreationRateXrt`
  - The throughput of monogesting operations in terms of xRT.
- `mediaCreationRatePerSec`
  - The throughput of monogesting operations in terms of completed operations per second.

Example of response data:

```
<computeMetrics>
```

```

<errorsOccurred>false</errorsOccurred>
<mediaOperationLoad>48.0</mediaOperationLoad>
<mediaOperationRateXrt01Min>458.0</mediaOperationRateXrt01Min>
<mediaOperationRateXrt05Min>229.0</mediaOperationRateXrt05Min>
<mediaOperationRateXrt15Min>114.5</mediaOperationRateXrt15Min>
<mediaOperationRatePerSec01Min>15.0</mediaOperationRatePerSec01Min>
<mediaOperationRatePerSec05Min>7.5</mediaOperationRatePerSec05Min>
<mediaOperationRatePerSec15Min>3.75</mediaOperationRatePerSec15Min>
<monogestCreationLoad>16.0</monogestCreationLoad>
<monogestCreationRateXrt01Min>458.0</monogestCreationRateXrt01Min>
<monogestCreationRateXrt05Min>229.0</monogestCreationRateXrt05Min>
<monogestCreationRateXrt15Min>114.5</monogestCreationRateXrt15Min>
<monogestCreationRatePerSec01Min>23.0</monogestCreationRatePerSec01Min>
<monogestCreationRatePerSec05Min>11.5</monogestCreationRatePerSec05Min>
<monogestCreationRatePerSec15Min>5.75</monogestCreationRatePerSec15Min>
</computeMetrics>

```

## Search Metrics

- `conventionalSearchLoad` & `lssSearchLoad`
  - The total number of search requests being serviced in the scope. Note that at system scope, load numbers are summed across nodes. For example, if each of five Data Nodes are working on a particular search operation, the load metric will be five rather than one.
- `conventionalSearchRateXrt` & `lssSearchRateXrt`
  - The throughput of search operations in terms of xRT.
- `conventionalSearchMediaPerSec` & `lssSearchMediaPerSec`
  - The throughput of search operations in terms of media files searched per second.

Example of response data:

```

<searchMetrics>
<errorsOccurred>false</errorsOccurred>
<conventionalSearchLoad>16.0</conventionalSearchLoad>
<conventionalSearchRateXrt01Min>586043.0</conventionalSearchRateXrt01Min>
<conventionalSearchRateXrt05Min>293022.5</conventionalSearchRateXrt05Min>
<conventionalSearchRateXrt15Min>146510.75</conventionalSearchRateXrt15Min>
<conventionalSearchRateMediaPerSec01Min>64.0</conventionalSearchRateMediaPerSec01Min>
<conventionalSearchRateMediaPerSec05Min>32.0</conventionalSearchRateMediaPerSec05Min>
<conventionalSearchRateMediaPerSec15Min>16.0</conventionalSearchRateMediaPerSec15Min>
<lssSearchLoad>4096.0</lssSearchLoad>
<lssSearchRateXrt01Min>4.598674E7</lssSearchRateXrt01Min>
<lssSearchRateXrt05Min>2.299337E7</lssSearchRateXrt05Min>
<lssSearchRateXrt15Min>1.1496685E7</lssSearchRateXrt15Min>
<lssSearchRateMediaPerSec01Min>30584.0</lssSearchRateMediaPerSec01Min>

```

```
<IssSearchRateMediaPerSec05Min>15292.0</IssSearchRateMediaPerSec05Min>
<IssSearchRateMediaPerSec15Min>7646.0</IssSearchRateMediaPerSec15Min>
</searchMetrics>
```

---

## Disaster Recovery Info Metrics

Disaster Recovery Info Metrics returns errorsOccurred and a single, formatted string designed to be easily machine parsable, which contains the metrics.

### Status

- `Enabled & Role & Replication Paused`
  - These settings control if the Disaster Recovery feature is activated and currently running.

### Trigger Configuration

- `Replication Frequency (seconds)`
  - This shows how often the system will automatically check for new media to replicate.
- `Maximum Backlog Count Before Replication`
  - This show how large the backlog of media waiting to be replicated can be before replication will be trigger ahead of the next scheduled run.

### Metrics Scope Report

- `Nodes`
  - Count of Data Nodes that are being asked for Disaster Recovery metrics.

### Awaiting Replication

- `Total Media Bundles`
  - Current count of media bundles in queue to be replicated.
- `Average Media Bundles Per Data Node`
  - Average media bundles on each Data Node in queue to be replicated.
- `Oldest Media Queued Age (ddd:hh:mm:ss)`
  - Total time since media declaration for the oldest media that is awaiting replication, in days:hours:minutes:seconds.

## Errors

- `Nodes not responding to metrics query`
  - Count of Data Nodes that did not respond to the current metrics query.
  - If this is zero, then `errorsOccurred` will be false, otherwise `errorsOccurred` will be true. (See the [Errors](#) subsection under [Metrics Entities](#) above for a description of `errorsOccurred`.)
- `Nodes with recent replication errors`
  - Count of Data Nodes that have had recent replication errors.
- `Recent Replication Errors`
  - Count of recent replication errors in last X minutes.
  - Covers `Last 1 Minute`, `Last 5 Minutes`, and `Last 15 Minutes`

## Replications (media bundles)

- `Last 1 Minute & Last 5 Minutes & Last 15 Minutes`
  - Total number of replications completed in last X minutes.

## Replication rate (media bundles per minute)

- `Last 1 Minute & Last 5 Minutes & Last 15 Minutes`
  - Rate of replications completed in last X minutes.

## Replication Latency (average, seconds since media declaration)

- `Last 1 Minute & Last 5 Minutes & Last 15 Minutes`
  - Average time for replications to occur in last X minutes, based upon the time the media was declared.

## Example of response data:

```
<disasterRecoveryMetricsInfo>
  <errorsOccurred>false</errorsOccurred>
  <disasterRecoveryMetricsInfoStr>Status\n\tEnabled: true\n\tRole: primary\n\tReplication Paused: false\n\tTrigger Configuration\n\tReplication Frequency (seconds): 1\n\tMaximum Backlog Count Before Replication: 1\n\tMetrics Report Scope\n\tNodes: 2 (aggregated statistics)\n\tAwaiting Replication\n\tTotal Media Bundles: 14284\n\tAverage Media Bundles per Data Node: 7142\n\tOldest Media Queued Age (ddd:hh:mm:ss): 000:00:01:49\n\tErrors\n\tNodes not responding to metrics query: 0 of 2\n\tNodes with recent replication errors: [none]\n\tReplication Errors\n\t\tLast 1 minute: 0\n\t\tLast 5 minutes: 0\n\t\tLast 15 minutes: 0\n\t\tSince start-up: 0\n\tReplications (media bundles)\n\t\tLast 1 minute: 2798\n\t\tLast 5 minutes: 6818\n\t\tLast 15 minutes: 6818\n\t\tSince start-up: 6818\n\tReplication Rate (media bundles per minute)\n\t\tLast 1 minute: 2411.422\n\t\tLast 5 minutes: 1029.
```

```
880\n\tLast 15 minutes: 403.992\nReplication Latency (average, seconds since media declaration)\n\tLast 1
minute: 93.307\n\tLast 5 minutes: 64.644\n\tLast 15 minutes: 64.644\n</disasterRecoveryMetricsInfoStr>
</disasterRecoveryMetricsInfo>
```

---

## Other Entities

---

### Media Batch Drop Info

---

Example of media batch drop in progress:

```
<mediaBatchDrop>
<status>InProgress</status>
<errorsOccurred>>false</errorsOccurred>
</mediaBatchDrop>
```

Example of a completed media batch drop:

```
<mediaBatchDrop>
<status>Completed</status>
<errorsOccurred>>false</errorsOccurred>
<mediaBatchDropResult href="http://example/mediabatchdropresult"/>
</mediaBatchDrop>
```

Once all drops in the batch are complete, the status element contains the value "Completed" and a `mediaBatchDropResult` element is available.

An `errorsOccurred` value of true could have two causes:

- The Search Grid application encountered internal errors.
- The Begin Media Batch Drop request referenced media that were unknown to the system. This can occur if the media were previously dropped or were never declared. This condition applies only if the Begin Media Batch Drop request set strict to true (or left it unspecified).

If `errorsOccurred` is true, then the client should call Get Media Batch Drop Result (via the URI in the `mediaBatchDropResult` element) to determine which media were successfully dropped. If `errorsOccurred` is false, then the client may skip retrieving the `mediaBatchDropResult` and invoke End Media Batch Drop.

---

## Segments and Classifiers

---

### Segments

---

In Search Grid, a *segment* represents a time-based region associated with a piece of media. A segment comprises four pieces of information:

---

[https://wem-dochub.mindtouch.us/Nexidia\\_Analytics\\_and\\_Data\\_Exchange\\_Framework/NXQC\\_12.5/Grid\\_Documentation/NiC...](https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentation/NiC...)

Updated: Thu, 30 Apr 2026 18:17:39 GMT

Powered by  mindtouch™



- The start time of the segment relative to the start of the media.
- The end time of the segment relative to the start of the media.
- The *type* of the segment, a 32-bit signed integer that defines its meaning.
- The segment's *value*, a 32-bit signed integer conveying detailed information. The meaning of the segment value depends on the segment's type.

Segments are generally addressed in aggregations called *segment batches*. All segments in a batch share the same segment type.

NiCE products produce segment types in the range 0-8192, inclusive. Client applications may use segment types above 8192 for information specific to the client application's domain. For example, a client application in the video production industry could define segment type 9444 to indicate a commercial break.

---

## Classifiers

Each Search Grid classifier produces segments of a specific type.

### Speech, Type 4010

The speech classifier identifies areas of the audio with and without speech. Segments without speech may contain silence, non-speech noise, music, or speech too quiet for the classifier to detect. In speech segments, segment value 1 denotes a speech, while segment value 0 denotes non-speech.

When the speech classifier is used, a handful of optional settings can control its behavior:

- Minimum Silence Interval: The shortest duration of silence the classifier will admit as a silence segment. Silence intervals less than this duration will be merged into adjacent speech segments.
- Minimum Speech Duration: The shortest duration of speech the classifier will admit as a speech segment. Speech utterances below this duration will be merged into adjacent silence segments.
- Flare Width: The duration of silence to count as speech at the start and end of "true" speech segments. If flare causes speech segments to overlap, they will be merged to a single segment.

### Gender, Type 4600

The gender classifier identifies a speaker's gender as male or female. Note that all regions of the audio—even silent regions—are classified as one gender or the other. Therefore, it is often useful to combine gender classification with speech classification. In gender segments, segment value 0 denotes male speech, while segment value 1 denotes female speech.

## DTMF, Type 5030

The DTMF classifier identifies tones used in dual-tone multi-frequency (DTMF) signaling, a system used in touch-tone telephones. In DTMF segments, the segment value indicates the meaning of the DTMF tone as shown in the table below. Note that the numbering system used for DTMF segment values may seem counterintuitive; for example, the segment value for DTMF tone "4" is 5 rather than 4. This is because the segment value numbering is based on the layout of a standard 4x4 DTMF keypad.

| DTMF Tone | Segment Value |
|-----------|---------------|
| 0         | 14            |
| 1         | 1             |
| 2         | 2             |
| 3         | 3             |
| 4         | 5             |
| 5         | 6             |
| 6         | 7             |
| 7         | 9             |
| 8         | 10            |
| 9         | 11            |
| A         | 4             |
| B         | 8             |
| C         | 12            |

| DTMF Tone | Segment Value |
|-----------|---------------|
| D         | 16            |
| #         | 13            |
| *         | 15            |
| Dial tone | 17            |
| Ring      | 18            |
| Busy      | 19            |
| No tone   | 20            |

### Diarization, Type 6000

Diarization distinguishes between two speakers in a single-channel recording. This is especially useful for call recorders that do not produce channel-separated audio. In diarization segments, segment value 1 indicates the first speaker and value 2 indicates the second speaker. The value 0 indicates that no speaker was detected. Note that diarization detects only two speakers; if used on recordings with more than two speakers, it will produce inaccurate results.

### Application-Defined, Types 8193-9999

Values in this range can be used as custom application-defined types. Typically, this will be used in the context of multi-channel media operations.

---

## Languages for Language Identification

The language identification API expresses languages as a 32-bit signed integer called a language ID. Language identification requests use language ID values to specify a set of languages to identify in the audio. Several factors influence the set of languages to request:

- Use only languages configured for language identification in the Search Grid instance. Most language models contain only a subset of the supported languages, so consult your administrator to determine the supported set. Note that the set of languages available for language identification is not necessarily the same as the set of installed language packs.

- Favor small sets of possible languages. Increasing the number of languages in the request decreases the accuracy of the resulting scores. Therefore, the ideal number of languages for a single language identification request is two. Accuracy drops off rapidly when identifying ten or more languages.
- Avoid identifying multiple dialects of the same language. For example, identifying Australian English, North American English, and UK English in the same operation is unlikely to produce accurate results.

For each requested language, Search Grid produces a confidence score in the range 0-100. Interpretation of the scores depends on several factors, including audio quality, the number of requested languages, and the phonetic similarity of the requested languages. In most situations, the client application should simply treat the language with the highest confidence score as the language used in the audio. Languages not supported by the configured language identification model will produce a score of zero.

Language models produced by NiCE use a consistent set of language ID values across language packs and language identification models. The language ID of a language pack is available via the Get Language Pack Info operation. Also, the Get Language Packs for Language operation can show what language packs correspond to a given language ID. The following table lists NiCE's standard language ID values:

| Language                | ID |
|-------------------------|----|
| Arabic, East Maghrebi   | 28 |
| Arabic, Egyptian        | 2  |
| Arabic, Gulf            | 12 |
| Arabic, Levantine       | 13 |
| Arabic, Modern Standard | 3  |
| Arabic, North Levantine | 22 |
| Croatian                | 21 |
| Dari                    | 23 |
| Dutch                   | 15 |

| Language                | ID |
|-------------------------|----|
| English, Australian     | 27 |
| English, North American | 0  |
| English, UK             | 9  |
| Farsi                   | 11 |
| French, Canadian        | 31 |
| French, European        | 29 |
| German                  | 8  |
| Hebrew                  | 25 |
| Hindi                   | 17 |
| Indonesian              | 24 |
| Iraqi                   | 4  |
| Italian                 | 33 |
| Japanese                | 6  |
| Korean                  | 5  |
| Mandarin                | 10 |

| Language                | ID |
|-------------------------|----|
| Pashto                  | 14 |
| Russian                 | 18 |
| Serbian                 | 20 |
| Somali                  | 32 |
| Sorani                  | 7  |
| Spanish, Castilian      | 30 |
| Spanish, Latin American | 1  |
| Tagalog                 | 26 |
| Thai                    | 19 |
| Urdu                    | 16 |



Customers using Nexidia's Language ID Training Tool product may use ID values other than the ones listed in the above table. However, using non-standard language ID values is discouraged because it breaks any direct association with language packs. Specifically, using non-standard language ID values prevents the Get Language Packs for Language operation from returning useful results.

## Structured Queries

A structured query is an advanced technique for searching multiple terms with complex constraints. The process of creating a structured query is much like developing software. First, the query is written in a specialized query language analogous to a source file written in C# or Java. Then the structured query text is *compiled* to a binary form optimized for searching a user's specific audio content. Results from structured query searches can be analyzed to help tune the query structure.

The process of creating an effective structured query is quite involved, and their use is not recommended for most client applications.

---

## Structured Query Operations

- [Validate Structured Query](#)
- [Begin Compilation](#)
- [Get Compilation Info](#)
- [End Compilation](#)
- [Get Compilation Result Batch](#)
- [Delete Compilation Result Batch](#)
- [Get Structured Query Inventory](#)
- [Add Structured Query](#)
- [Find Structured Query](#)
- [Get Structured Query Info](#)
- [Get Structured Query Image](#)
- [Delete Structured Query](#)

---

## Validate Structured Query

Identifies problems with structured query text, including syntax errors and warnings. It is recommended to use this operation during structured query design to fix problems before attempting to compile the query.

For valid queries, validation results include a set of `queryExpression` elements that indicate the structure of the query from the phonetic engine's perspective. See the [Query Expressions](#) section below for more details about how to interpret query expressions.

Request entities using the text/plain content type must conform to the standards for its use in HTTP, including the following:

- Consistent with RFC 2616, text/plain entities transmitted over HTTP may use CRLF, bare CR, or bare LF to signify line endings.
- The character set is assumed to be US-ASCII unless the Content-Type header specifies a different character set.

Therefore, requests with structured query text containing characters outside the US-ASCII range *must* explicitly specify an appropriate character set such as UTF-8.

- Multi-byte Unicode encodings must include byte order, either with a character set that specifies byte order (e.g. UTF-16BE or UTF-32LE) or with a byte order mark in the request entity.

Note that unlike Validate Search Term, Validate Structured Query is a POST operation, not GET.

All authenticated users have authorization to invoke this operation.

## Request (As Text)

```
POST [Structured Query Validation URI provided by Language Pack Info] HTTP/1.1
Content-Type: text/plain
Accept: application/vnd.nexidia.searchgrid+xml

Result = Before_10("My name is", \
    "How may I help you?")
```

## Request (As XML)

```
POST [Structured Query Validation URI provided by Language Pack Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml

<queryText>
Result = Before_10("My name is", \
    "How may I help you?")
</queryText>
```

## Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.searchgrid+xml

[Example response entities below]
```

Example response entity for a structured query with no syntax warnings or errors:

```
<structuredQueryValidation>
<queryText>
Result = Before_10("My name is", \
    "How may I help you?")
</queryText>
<queryExpression id="1">
<name>Result</name>
<nameLine>1</nameLine>
<nameColumn>1</nameColumn>
<operator>BEFORE</operator>
<operatorLine>1</operatorLine>
```



```

<operatorColumn>10</operatorColumn>
<windowMs>10000</windowMs>
<argument>
  <queryExpression ref="3"/>
  <weight>1.0</weight>
</argument>
<argument>
  <queryExpression ref="2"/>
  <weight>1.0</weight>
</argument>
<isUsed>true</isUsed>
</queryExpression>
<queryExpression id="2">
  <name>#1</name>
  <operator>TERM</operator>
  <operatorLine>2</operatorLine>
  <operatorColumn>20</operatorColumn>
  <term>How may I help you?</term>
  <parentExpression ref="1"/>
  <isUsed>true</isUsed>
</queryExpression>
<queryExpression id="3">
  <name>#0</name>
  <operator>TERM</operator>
  <operatorLine>1</operatorLine>
  <operatorColumn>20</operatorColumn>
  <term>My name is</term>
  <parentExpression ref="1"/>
  <isUsed>true</isUsed>
</queryExpression>
<validForCompilation>true</validForCompilation>
</structuredQueryValidation>

```

Example response entity for a structured query with syntax warnings but no errors:

```

<structuredQueryValidation>
  <queryText>
    Result = Before_10("My", \
      "How may I help you?")
  </queryText>
  <queryExpression id="1">
    <name>Result</name>
    <nameLine>1</nameLine>
    <nameColumn>1</nameColumn>
    <operator>BEFORE</operator>
    <operatorLine>1</operatorLine>
    <operatorColumn>10</operatorColumn>
    <windowMs>10000</windowMs>
    <argument>
      <queryExpression ref="3"/>
      <weight>1.0</weight>
    </argument>
    <argument>

```

```

    <queryExpression ref="2"/>
    <weight>1.0</weight>
  </argument>
  <isUsed>true</isUsed>
</queryExpression>
<queryExpression id="2">
  <name>#1</name>
  <operator>TERM</operator>
  <operatorLine>2</operatorLine>
  <operatorColumn>20</operatorColumn>
  <term>How may I help you?</term>
  <parentExpression ref="1"/>
  <isUsed>true</isUsed>
</queryExpression>
<queryExpression id="3">
  <name>#0</name>
  <operator>TERM</operator>
  <operatorLine>1</operatorLine>
  <operatorColumn>20</operatorColumn>
  <term>My</term>
  <parentExpression ref="1"/>
  <isUsed>true</isUsed>
</queryExpression>
<syntaxWarning>
  <warningCode>PhraseTooShort</warningCode>
  <line>1</line>
  <column>20</column>
</syntaxWarning>
<validForCompilation>true</validForCompilation>
</structuredQueryValidation>

```

Example response entity for a structured query with syntax errors:

```

<structuredQueryValidation>
  <queryText>
Result = Before_10("My", \
    "How may I help you?"
  </queryText>
  <syntaxWarning>
    <warningCode>PhraseTooShort</warningCode>
    <line>1</line>
    <column>20</column>
  </syntaxWarning>
  <syntaxError>
    <errorCode>UnmatchedParenthesis</errorCode>
    <line>1</line>
    <column>24</column>
  </syntaxError>
  <validForCompilation>false</validForCompilation>
</structuredQueryValidation>

```

- A response of 200 (OK) does not imply that the structured query text is valid, only that validation was executed successfully.

- Line and column numbers refer to positions in the query text as submitted in the request, not in the response entity.
- The line and column elements are optional, and may not appear with some error or warning codes.

```
HTTP/1.1 404 Not Found
x-nx-error-code: UnknownLanguagePack
x-nx-error-resource: [The request URI]
```

## Syntax Warning and Error Codes

### Warnings:

- PhraseTooShort
- OrTooManyArguments
- TimeOperandTooShort
- TimeOperandTooLong
- PossibleAcronym
- DuplicateSearchTerm

### Errors:

- NoExpressionsFound
- InvalidExpressionName
- DuplicateExpressionName
- IncompleteExpression
- MissingOperator
- InvalidOperator
- UnmatchedQuote
- WeightNotAllowed
- InvalidWeight
- MissingTimeParameter
- InvalidTimeParameter

- MissingCountParameter
- InvalidCountParameter
- MissingArgumentList
- InvalidArgument
- UnmatchedParenthesis
- TooFewArguments
- TooManyArguments
- TokensAfterExpression
- ReferencedExpressionNotDefined
- InvalidLineContinuationChar
- InvalidIncludeDirective
- CannotAccessIncludeFile
- MissingArgumentDelimiter
- InvalidInvertDirective
- InvalidMaxResultsDirective
- InvalidDtmfSequence

---

## Query Expressions

The set of `queryExpression` elements for a given structured query are the query's *parse tree*; that is, how the phonetic engine internally interprets the query. The response entity presents the query expressions as a flat list, but expressions contain references to one another that describe a treelike structure. Query expressions contain the following attributes and sub-elements:

- `id`: Identifies the query expression if it is referenced as an argument of another query expression.
- `name`: The name assigned to the expression in the query text. If no name was supplied, the phonetic engine generates one.
- `nameLine`: The one-indexed line number of the expression name in the original structured query text. Omitted if the expression does not have a name.

- `nameColumn`: The one-indexed column number of the expression name in the original structured query text. Omitted if the expression does not have a name.
- `operator`: Identifies the kind of expression. Refer to the [Structured Query Syntax](#) section for details about the supported operators.
- `operatorLine`: The one-indexed line number of the operator in the original structured query text. This member may be omitted in some rare cases.
- `operatorColumn`: The one-indexed column number of the operator in the original structured query text. This member may be omitted in rare cases.
- `windowMs`: Describes the time basis for operators that use one. Present only for AND, ANDNOT, SUBSET, BEFORE, FIRST, and LAST operators.
- `count`: Parameter for the SUBSET operator; omitted for all other operators.
- `argument`: Identifies a query expression that is an input into this one. The document order of argument elements implies their order in the operator.
  - `ref`: Corresponds to the `id` attribute of the referenced `queryExpression` element.
  - `weight`: The relative contribution of the argument.
- `parentExpression`: The `ref` attribute corresponds to the `id` attribute of an expression that consumes this expression as an argument. A given expression may have zero or more parent expressions.
- `inUse`: True if the expression contributes to query execution; false otherwise.

---

## Begin Compilation

Client applications should verify structured query correctness with the Validate Structured Query operation prior to invoking Begin Compilation.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

```
POST [Compilation Inventory URI provided by Tenant Info] HTTP/1.1
Content-Type: application/vnd.nexidia.searchgrid+xml
Accept: application/vnd.nexidia.searchgrid+xml
```

[See Compilation Entities section for Compilation Info entity specification]

## Response

HTTP/1.1 201 Created  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Location: [Specific Compilation URI]

[See Compilation Entities section for Compilation Info entity specification]

HTTP/1.1 400 Bad Request  
x-nx-error-code: InvalidStructuredQueryText  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: DuplicateStructuredQueryTextId  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: TooManyStructuredQueryTexts  
x-nx-error-resource: [The request URI]

HTTP/1.1 403 Forbidden  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

---

## Get Compilation Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Compilation URI provided by Begin Compilation operation] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Compilation Entities section for entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownCompilation  
x-nx-error-resource: [The request URI]

---

## End Compilation

Clients should invoke this operation immediately after the final compilation result batch is retrieved or the compilation operation has ended (e.g. due to an error). The server may reject new compilation operations or searches if too many

operations are underway.

After the End Compilation operation is complete, the specified Compilation resource is no longer available to clients.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

DELETE [Compilation URI provided by Begin Compilation operation] HTTP/1.1

---

## Response

HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownCompilation  
x-nx-error-resource: [The request URI]

---

## Get Compilation Result Batch

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Compilation Result Batch URI provided by Compilation Info or Search Info] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

[See Compilation Entities section for entity specification]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownSearchResultBatch  
x-nx-error-resource: [The request URI]

---

## Delete Compilation Result Batch

Clients should invoke this operation immediately after the compilation result batch is successfully retrieved. Note that the server may delay the compilation operation if too many result batches are left on the server.

Only the tenant user for the relevant tenant may invoke this operation.

## Request

DELETE [Compilation Result Batch URI provided by Compilation Info] HTTP/1.1

## Response

HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownSearchResultBatch  
x-nx-error-resource: [The request URI]

---

## Get Structured Query Inventory

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Structured Query Inventory URI] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml

```
<structuredQueryInventory>
  <structuredQuery href="http://example/structuredquery1"/>
  <structuredQuery href="http://example/structuredquery2"/>
  <structuredQuery href="http://example/structuredquery3"/>
</structuredQueryInventory>
```

---

## Add Structured Query

Client applications should record the value of the `id` attribute of the returned response entity. The `id` value is used to express searches (see [Search Entities](#) section).

The `languagePackGuid` supplied in the request must match the language pack used to create the structured query.

If an `externalStructuredQueryId` value is omitted, the lowercase string value of the structured query's internal GUID will be used. User-specified `externalStructuredQueryId` values must have a minimum length of 1 and a maximum length of 260; values may not contain leading or trailing whitespace. Values used for `externalStructuredQueryId` must be unique and invariant across all structured queries for the specified tenant.

The contents of the `queryText` element represent the text stored in the compiled structured query image, and may be



formatted differently than the original text that was compiled. The `queryText` uses UNIX-style line-endings, i.e. LF only, so it may need to be converted to the client platform's native line-endings before presentation to a user.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request (With Default `externalStructuredQueryId`)

```
POST [Structured Query Inventory URI provided by Tenant Info]?languagePackGuid=<languagePackGuid>
HTTP/1.1
Content-Type: application/octet-stream
Accept: application/vnd.nexidia.searchgrid+xml

[Entity is structured query image]
```

### Request (With Custom `externalStructuredQueryId`)

```
POST [Structured Query Inventory URI provided by Tenant
Info]?externalStructuredQueryId=<externalStructuredQueryId>&languagePackGuid=<languagePackGuid>
HTTP/1.1
Content-Type: application/octet-stream
Accept: application/vnd.nexidia.searchgrid+xml

[Entity is structured query image]
```

### Responses

```
HTTP/1.1 201 Created
Content-Type: application/vnd.nexidia.searchgrid+xml
Location:http://example/structuredquery1

<structuredQuery id="1">
  <creationDateTime>2010-05-30T09:30:10Z</creationDateTime>
  <externalStructuredQueryId>9e825c18-497c-47af-a989-8531aa72f756</externalStructuredQueryId>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <queryText>
Result = Before_10("My name is", \
                  "How may I help you?")
  </queryText>
</structuredQuery>
```

```
HTTP/1.1 400 Bad Request
x-nx-error-code: InvalidExternalId
x-nx-error-resource: [The request URI]
```

```
HTTP/1.1 409 Conflict
x-nx-error-code: DuplicateStructuredQueryId
x-nx-error-resource: [The existing structured query URI]
```

```
HTTP/1.1 403 Forbidden
x-nx-error-code: IncompatibleLanguagePack
x-nx-error-resource: [The request URI]
```

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownLanguagePack  
x-nx-error-resource: [The request URI]

HTTP/1.1 400 Bad Request  
x-nx-error-code: InvalidStructuredQuery  
x-nx-error-resource: [The request URI]

---

## Get Structured Query Info

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

GET [Structured Query URI provided by Structured Query Inventory or Find Structured Query] HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Response

HTTP/1.1 200 OK  
Content-Type: application/vnd.nexidia.searchgrid+xml  
Accept: application/vnd.nexidia.searchgrid+xml

```
<structuredQuery id="101">  
  <creationDateTime>2010-05-30T09:30:10Z</creationDateTime>  
  <externalStructuredQueryId>Agent Compliance v2</externalStructuredQueryId>  
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>  
  <queryText>  
Result = Before_10("My name is", \  
    "How may I help you?")  
  </queryText>  
</structuredQuery>
```

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownStructuredQuery  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedStructuredQuery  
x-nx-error-resource: [The request URI]

---

## Get Structured Query Image

Returns the stored structured query image.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

## Request

GET [Structured Query URI provided by Structured Query Inventory or Find Structured Query] HTTP/1.1  
Accept: application/octet-stream

## Response

HTTP/1.1 200 OK  
Content-Type: application/octet-stream

[Entity is the structured query image]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownStructuredQuery  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedStructuredQuery  
x-nx-error-resource: [The request URI]

---

## Find Structured Query

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request (By Structured Query ID)

GET [Structured Query Inventory URI]?structuredQueryId=<structuredQueryId> HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

### Request (By External Structured Query ID)

GET [Structured Query Inventory URI]?externalStructuredQueryId=<externalStructuredQueryId> HTTP/1.1  
Accept: application/vnd.nexidia.searchgrid+xml

## Response

HTTP/1.1 301 Moved Permanently  
Location: [Specific Structured Query URI]

HTTP/1.1 404 Not Found  
x-nx-error-code: UnknownStructuredQuery  
x-nx-error-resource: [The request URI]

HTTP/1.1 410 Gone  
x-nx-error-code: DeletedStructuredQuery  
x-nx-error-resource: [The request URI]

---

## Delete Structured Query

After completion, subsequent calls to the deleted Structured Query URI will result in 410 (Gone) or 404 (Not Found) responses. Subsequent searches referencing the deleted structured query will fail at the Begin Search operation. In-progress searches that reference the deleted structured query may complete normally or may complete with errors.

Only the tenant user for the relevant tenant has authorization to invoke this operation.

### Request

DELETE [Structured Query URI provided by Structured Query Inventory or Find Structured Query] HTTP/1.1

---

## Response

HTTP/1.1 204 No Content

---

## Compilation Entities

---

### Compilation Request

Every compilation request contains two elements:

- The compilation space defines the set of media against which to compile.
- The compilation expression defines the queries to compile.

---

### Compilation Space

A compilation request contains exactly one `compilationSpace` element. The `compilationSpace` element's content defines the set of media to be searched:

- exactly one `languagePack` element
  - Note that language pack GUIDs are available via the Get Language Pack Info operation as well as from the manifest.xml file installed with the language pack.
- at most one of the following media batch representations:
  - `mediaList` element
  - `invariantData` element

---

## All Media for Language Pack

Defines the compilation space as the set of the tenant's media for which pats are available for the specified language pack.

```
<compilationSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
</compilationSpace>
```

---

## Media List

A media list contains an explicit list of media IDs to use as input for compilation.

See [Media Batch Representations](#) for a detailed description of media lists.

```
<compilationSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <mediaList>
    <media ref="1001"/>
    <media ref="1002"/>
    <media ref="1003"/>
  </mediaList>
</compilationSpace>
```

---

## Invariant Data

Each language pack includes a set of invariant data (also called "normalization data" or "default normalization data") that can be used for compilation. Compiling against invariant data permits the creation of structured queries before Search Grid has created any pats. However, a structured query compiled against invariant data will produce less accurate results than a query compiled against client-supplied media.

```
<compilationSpace>
  <languagePack guidref="966da303-bde8-4d9d-83e4-d2c49af6ae45"/>
  <invariantData/>
</compilationSpace>
```

---

## Compilation Expression

Each compilation request must contain the following top-level element:

- `compilationExpression`

---

## Structured Query Text

Each `structuredQueryText` 's `id` attribute must be an integer that is unique within the `compilationRequest`. The `id` value is used to correlate result and error information with the structured query text. Values must be between 1 and  $2^{31} - 1$ , inclusive. Values within a compilation request do not need to be consecutive.

Structured query text may contain the `<` and `&` characters, which are illegal in XML documents. Client applications must ensure that such characters are encoded appropriately for XML. Options include escaped entities (`<` and `&`, respectively) or a CDATA section.

See [Structured Query Syntax](#) section for details about the syntax of structured query text.

```
<structuredQueryText id="1">
Result = Before_10("My name is", \
    "How may I help you?")
</structuredQueryText>
```

---

## Compilation Expression

A `compilationExpression` element must contain one or more `structuredQueryText` elements. Note that each contained `structuredQueryText` element must have an `id` attribute value that is unique within the `compilationRequest`.

```
<compilationExpression>
<structuredQueryText id="1">
Result = Before_10("My name is", \
    "How may I help you?")
</structuredQueryText>
<structuredQueryText id="2">
Result = OR("speech analytics", "phonetic search")
</structuredQueryText>
</compilationExpression>
```

---

## Compilation Info

```
<compilation>
<status>Completed</status>
<errorsOccurred>>false</errorsOccurred>
<progress>
  <mediaCountProcessed>64204</mediaCountProcessed>
  <mediaDurationProcessedMs>594903583</mediaDurationProcessedMs>
</progress>
<compilationResultBatchInventory>
  <compilationResultBatch href="http://example/compilationresultbatch1"/>
</compilationResultBatchInventory>
</compilation>
```

If `errorsOccurred` is true, at least one error occurred during the compilation operation, and available results may have been impacted.

"Processed" means "processed without error".

---

## Compilation Result Batch

Each `compilationResultBatch` element contains one or more `compilationResult` elements.

Each `structuredQuery` element contains a base 64 encoded binary query.

```
<compilationResultBatch>
  <compilationResult>
    <!-- exactly one structuredQueryText per compilationResult -->
    <structuredQueryText ref="1"/>
    <structuredQuery>EhA=</structuredQuery>
  </compilationResult>
  <compilationResult>
    <!-- exactly one structuredQuery per compilationResult -->
    <structuredQueryText ref="2"/>
    <structuredQuery>EhA=</structuredQuery>
  </compilationResult>
</compilationResultBatch>
```

---

## Structured Query Search Entities

---

### Search Query

The `searchQuery` element specifies a search against a specific structured query. Structurally, a `searchQuery` can be used anywhere a `searchTerm` can be used, although the `searchAlgorithm` must be "conventional". Each `searchQuery`'s `id` attribute value must be an integer that is unique across `searchQuery` elements in the `searchRequest`. The `id` value is used to correlate result and error information with the search term. Values must be between 1 and  $2^{31} - 1$ , inclusive. Values within a `searchRequest` do not need to be consecutive. Structured queries may either be stored in Search Grid (via the Add Structured Query operation) or provided as base 64 encoded binary form. If a query is going to be used over a long period of time, it is more efficient to store the query in Search Grid than to pass the query in binary form for each search. If the threshold or false alarm rate is not provided, the default false alarm rate of 0.1 false alarms per media hour will be used.

```
<searchQuery id="1">
  <!-- default threshold and stored structured query -->
  <structuredQuery ref="101"/>
</searchQuery>
<searchQuery id="2">
  <!-- explicit threshold and stored structured query -->
  <structuredQuery ref="102"/>
```

```
<threshold>26.0</threshold>
</searchQuery>
<searchQuery id="3">
  <!-- explicit false alarm rate and binary structured query -->
  <structuredQuery>EhA=</structuredQuery>
  <falseAlarmsPerHour>0.05</falseAlarmsPerHour>
</searchQuery>
```

---

## Structured Queries in Search Expressions

Search requests may contain `searchTerm` and `searchQuery` elements in the same request.

```
<searchExpression>
  <searchTerm id="1">
    <queryString>thank you for calling</queryString>
    <threshold>70.0</threshold>
  </searchTerm>
  <searchQuery id="1">
    <structuredQuery ref="101"/>
  </searchQuery>
  <operator>any</operator>
  <timeOperator>first</timeOperator>
  <timeBasisMs>60000</timeBasisMs>
</searchExpression>
```

---

## Structured Query Hits in Search Result Batches

Hits for structured queries reference a `searchQuery` rather than a `searchTerm`.

```
<searchResultBatch>
  <searchResult>
    <media ref="1001"/>
    <hit>
      <searchTerm ref="1"/>
      <score>89.3934</score>
      <startMs>34840</startMs>
      <endMs>34940</endMs>
    </hit>
    <hit>
      <searchQuery ref="1"/>
      <score>30.9473</score>
      <startMs>54840</startMs>
      <endMs>54940</endMs>
    </hit>
  </searchResult>
</searchResultBatch>
```



---

## Structured Query Result Analysis

Structured query result analysis provides details on how individual components of a structured query contribute to the overall result. To activate result analysis, set `analyzeQueryResults` to true in the `searchOptions` element. Note that result analysis produces a high volume of data, so search spaces should be kept as small as possible; 32 media is the maximum recommended size.

---

### Example Search Options

```
<searchOptions>
  <!-- new optional element, defaults to false -->
  <!-- valid only for conventional search -->
  <analyzeQueryResults>true</analyzeQueryResults>
  <searchAlgorithm>conventional</searchAlgorithm>
</searchRequest>
```

---

### Example Search Result Batch

```
<searchResultBatch>
  <searchResult>
    <media ref="1001"/>
    <hit>
      <searchQuery ref="1"/>
      <score>89.3934</score>
      <startMs>34840</startMs>
      <endMs>34940</endMs>
      <!-- present only if analyzeQueryResults was true and there was at least one searchQuery -->
      <!-- see Validate Structured Query operation for details about queryExpression elements -->
      <queryExpression id="1">
        <name>Result</name>
        <operator>BEFORE</operator>
        <windowMs>10000</windowMs>
        <argument>
          <queryExpression ref="3"/>
          <weight>1.0</weight>
        </argument>
        <argument>
          <queryExpression ref="2"/>
          <weight>1.0</weight>
        </argument>
      </queryExpression>
      <queryExpression id="2">
        <name>#1</name>
        <operator>TERM</operator>
        <term>How may I help you?</term>
      </queryExpression>
      <queryExpression id="3">
        <name>#0</name>
        <operator>TERM</operator>
```

```
<term>My name is</term>
</queryExpression>
<!-- present only if analyzeQueryResults was true and there was at least one searchQuery -->
<!-- one queryExpressionResult element for each queryExpression element -->
<queryExpressionResult>
  <queryExpression ref="1"/>
  <valid>true</valid>
  <contributor>true</contributor>
  <score>83.2345</score>
  <startMs>34840</startMs>
  <endMs>34940</endMs>
</queryExpressionResult>
</hit>
</searchResult>
</searchResultBatch>
```

---

## Structured Query Syntax

---

### Overview

This topic provides high-level information about Query syntax.

---

### File Format

Text query files (.NTQ) must be in ASCII text or UTF-8/UTF-16 Unicode text format. Unicode files require a byte order mark (BOM).

---

### Comments

The semicolon is used to indicate that the remainder of the line is a comment and should not be processed. Named

---

### Expressions

A named expression is any line in the Query file that is neither a comment nor an include statement must be a statement of the form:

```
| Name=<Expression>
```

The name expression can be any sequence of alphabetic characters, numeric characters or the underscore character (a-z, A-Z, 0-9, \_). Names are not case-sensitive.

---

### Phrase Expressions

A phrase expression is an expression that represents a phonetic search for a phrase. Any phrase that is normally accepted by the NiCE Phonetic Search Engine may be used, with the exception that the 'within' operator (the '&' in

English) is not supported in structured mining. Here is an example of a named phrase expression:

```
| MyPhrase="Nexidia"
```

## Operator Expressions

This topic provides information about Search Grid operators.

### Definition

An operator expression specifies a relationship among its arguments that must be satisfied.

### Expressions

The following table describes the operator expressions that are currently supported:

| Operator | Description                                                                                                                                                                                                    |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AND      | The arguments to this operator must all occur within a specified interval of time, or optionally, somewhere within the entire file.                                                                            |
| OR       | At least one of the arguments to this operator must occur.                                                                                                                                                     |
| ANDNOT   | The second and subsequent arguments to this operator must not occur within a specified interval of time, or optionally, somewhere within the entire file, before or after an occurrence of the first argument. |
| SUBSET   | At least the specified number of the arguments to this operator must occur within a specified interval of each other.                                                                                          |
| BEFORE   | The first argument to this operator must occur within a specified interval of time, or optionally, somewhere within the entire file, before an occurrence of the second argument.                              |

| Operator | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FIRST    | The argument to this operator must occur within a specified interval from the beginning of the file.                                                                                                                                                                                                                                                                                                                                                                                           |
| LAST     | The argument to this operator must occur within a specified interval from the end of the file.                                                                                                                                                                                                                                                                                                                                                                                                 |
| NOTFIRST | The argument to this operator must occur outside a specified interval from the beginning of the file.                                                                                                                                                                                                                                                                                                                                                                                          |
| NOTLAST  | The argument to this operator must occur outside a specified interval from the end of the file.                                                                                                                                                                                                                                                                                                                                                                                                |
| NOTLIKE  | Lets you improve accuracy by explicitly identifying common similar sounding phrases that can be confused with the desired target phrase. For example, you are looking for "prison", but you are getting results on "present", "president", and "presenting". Instead of just using "prison" as a phrase, you can instead use the NOTLIKE operator as: NOTLIKE("prison", "present", "president", "presenting"). This will help reduce false hits on phonetically similar, but distinct phrases. |

---

## Operator Arguments

This topic provides examples of selected operator expression and gives information about argument weights.

---

### Examples

Here is an example of an operator expression:

```
MyOperator=OR("L A", "Los Angeles")
```

This example is a named expression, MyOperator, that uses the OR operator to find occurrences of its arguments: "L A" and "Los Angeles."

In this example, the arguments in parentheses are unnamed phrase expressions. Any argument to an operator can be an unnamed phrase expression, an unnamed operator expression, or the name of a previously declared phrase or

operator named expression.

As an example of this, Expression1 and Expression2 below are equivalent:

```
Example1=AND_60("This", OR("That", "The Other"))
Example2a=OR("That", "The Other")
Example2=AND_60("This", Example2a)
```

---

## Argument Weights

Sometimes, it is important for an operator to depend on some arguments more than others. To implement this, each argument to an operator can be prefixed with a weight in square brackets. The weights that can be applied to the arguments to the **AND** and **OR** operators range from 0.0 to 1.0. The weights that can be applied to the arguments to the **ANDNOT** operator range from 0.0 to 10.0. The relative values of these weights determine the effect that each argument will have on the operator. So, for example, when all arguments have the same weight, it does not matter if they all have weight 1.0 or weight 0.5. But if one argument has weight 1.0 and another has weight 0.5, the first argument will have a much greater effect on the overall score for that operator. Unless the weight for an argument is explicitly given, its weight is 1.0.

Only the following operators accept weights:

- **AND** - All arguments can be weighted with a value of 0.0 to 1.0.
- **OR** - All arguments can be weighted with a value of 0.0 to 1.0
- **ANDNOT** - The second and subsequent arguments can be weighted with a value of 0.0 to 10.0 \* Here is an example of weighted arguments to an operator:

```
Example = OR([1.0]"this", [0.5]"That", [0.25]"The Other")
```

---

## Operator Parameters

This topic provides information about the parameters that are used with operators.

---

### Examples

Most operators require one or more parameters in addition to their arguments. For example, the **AND** operator takes a parameter to indicate how close together its arguments must occur. These parameters are appended to the operator, separated by an underscore character, as follows:

```
ParamExample=AND_60("This", "That")
```

In this example, "This" and "That" must occur within 60 seconds of each other.

The **SUBSET** operator takes two parameters:

```
TwoParams=SUBSET_2_60("This", "That", "The Other")
```

In this example, the SUBSET operator will search for two of the given arguments to appear within 60 seconds of each other.

---

## Line Continuation

Sometimes an expression may be too long to easily fit on a single line. In this case, the line-continuation character, backslash, can be used to indicate that the following line should be treated as an extension of the current one. For example: Sometimes an expression may be too long to easily fit on a single line.

```
ContinuedLine=OR("This", \  
"That", \  
"The Other")
```

---

## Spacing and Casing

White space and character casing is ignored, except for within quoted phrases. Thus, the following are equivalent:

```
Example=AND_60("This", "That")  
example = and_60 ("This", "That")
```

---

## Character Set

Expression names, operators and parameters all use English alphanumeric characters. Quoted phrases may use any character set supported by Unicode.

---

## Bidirectional behavior

To support right-to-left languages, the Unicode bidirectional formatting codes (LRE, LRO, RLE, RLO, PDF, LRM and RLM) are ignored within quoted phrases and can be used to enable easier presentation and editing of structured queries.



These characters are ignored completely, and are not treated as white space. Additional white space characters must be used to indicate word boundaries.

---

## Operator Restrictions

Like OR, the operators AND, ANDNOT, and BEFORE can be used without specifying a time window. In this case, they operate over the entire file being searched. You must be particularly careful when using this mode as it is very easy to get matches to component occurrences in completely unrelated parts of the audio.

In addition, the following restriction applies: A non-time-bound operator (no time window specified) cannot be used as input to a time-bound operator. For example, the following sequence is legal:

```
A = "hello"
B = "goodbye"
C = "may i help you"
greeting = BEFORE_10(A, C)
goodcall = BEFORE(greeting, B)
whereas the following is not legal:
greeting = BEFORE(A,B)
goodcall = BEFORE_10(greeting, C)
```

You must therefore use the non–time-bound operators as the final operators of a query (non time-bound can be used with other non–time-bound operators). This restriction does not apply to the OR operator, which may be used at any point. Output

The result of a query is the result of the last named expression in the file. Any expressions, named or unnamed, that do not affect the last expression are not processed.

---

## Operator Reference

This topic is designed to serve as a reference for using operators with NiCE Search Grid.

---

### OR

The following table provides information about the OR operator:

| Info Type  | Description                                      |
|------------|--------------------------------------------------|
| Example    | Name = OR (Arg1, Arg2,..., ArgN)                 |
| Parameters | None                                             |
| Arguments  | At least two                                     |
| Weights    | All arguments can be weighted                    |
| Semantics  | Used to find occurrences of any of the arguments |

---

### AND

The following table provides information about the AND operator:

| Info Type  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = AND_60 (Arg1, Arg2,..., ArgN)                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Parameters | Time window (optional), in integer seconds                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Arguments  | At least two                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Weights    | All arguments can be weighted                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Semantics  | Used to find audio segments in which all arguments occur within a given time window. If no time window is specified, the entire media duration of each file searched is used. You must be very careful with this option as you may get results for terms that occur in completely separate parts of the file. Note: Only returns high scores when all arguments have high scores within the time window. If even one argument does not have a high score, this operator will score very low. |

## ANDNOT

The following table provides information about the ANDNOT operator:

| Info Type  | Description                                             |
|------------|---------------------------------------------------------|
| Example    | Name = ANDNOT_60 (Arg1, Arg2,..., ArgN)                 |
| Parameters | Time window (optional), in integer seconds              |
| Arguments  | At least two                                            |
| Weights    | The second and subsequent arguments can be weighted     |
| Semantics  | Used to find audio segments in which the first argument |



| Info Type | Description                                                                                                                                                                                                                                                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           | <p>has a high score and all subsequent arguments do not. Like <b>AND</b> and <b>BEFORE</b>, the time argument is optional. If the time argument is not specified, the duration of the media file being searched will be used. Note: <b>ANDNOT_60(Arg1, Arg2, Arg3)</b> is not the same as <b>ANDNOT_60(Arg1, OR(Arg2, Arg3))</b>.</p> |

## SUBSET

The following table provides information about the SUBSET operator:

| Info Type  | Description                                                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = SUBSET_2_60 (Arg1, Arg2,..., ArgN)                                                                                                                                                                                                |
| Parameters | First: Number of arguments that must be found. Second: Time window, in integer seconds                                                                                                                                                   |
| Arguments  | At least two, and more than those specified in first parameter                                                                                                                                                                           |
| Weights    | Not supported                                                                                                                                                                                                                            |
| Semantics  | <p>This operator is effectively an AND operator that takes as input the highest-scoring subset of its arguments. Note: If fewer than the specified subset of arguments have high scores, this operator will return a very low score.</p> |

## BEFORE

The following table provides information about the BEFORE operator:

| Info Type  | Description                                                                                                                                                                                                                                                                                                                                                 |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = BEFORE_60 (Arg1, Arg2)                                                                                                                                                                                                                                                                                                                               |
| Parameters | Time window (optional), in integer seconds                                                                                                                                                                                                                                                                                                                  |
| Arguments  | Exactly two                                                                                                                                                                                                                                                                                                                                                 |
| Weights    | Not supported                                                                                                                                                                                                                                                                                                                                               |
| Semantics  | This operator is an AND operator that requires that the first argument come before the second. Like <a href="#">AND</a> and <a href="#">ANDNOT</a> , the time window is optional. If not specified, the duration of the file being searched is used. The non-time-bound version of this operator may not be used as an argument to any time-bound operator. |

---

## FIRST

The following table provides information about the FIRST operator:

| Info Type  | Description                                                                                                                      |
|------------|----------------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = FIRST_60 (Arg1)                                                                                                           |
| Parameters | Time window, in integer seconds, starting from the beginning of the file                                                         |
| Arguments  | Exactly one                                                                                                                      |
| Weights    | Not supported                                                                                                                    |
| Semantics  | This operator returns only the results from its argument that occur within the given time window from the beginning of the file. |

---

## LAST

The following table provides information about the LAST operator:

| Info Type  | Description                                                    |
|------------|----------------------------------------------------------------|
| Example    | Name = LAST_60 (Arg1)                                          |
| Parameters | Time window, in integer seconds, ending at the end of the file |
| Arguments  | Exactly one                                                    |
| Weights    | Not supported                                                  |

---

## NOTFIRST

The following table provides information about the NOTFIRST operator:

| Info Type  | Description                                                                                                                       |
|------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = NOTFIRST_60 (Arg1)                                                                                                         |
| Parameters | Time window, in integer seconds, starting from the beginning of the file                                                          |
| Arguments  | Exactly one                                                                                                                       |
| Weights    | Not supported                                                                                                                     |
| Semantics  | This operator returns only the results from its argument that occur outside the given time window from the beginning of the file. |

---

## NOTLAST

The following table provides information about the NOTLAST operator:

| Info Type  | Description                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------------------|
| Example    | Name = NOTLAST_60 (Arg1)                                                                                                    |
| Parameters | Time window, in integer seconds, ending at the end of the file                                                              |
| Arguments  | Exactly one                                                                                                                 |
| Weights    | Not supported                                                                                                               |
| Semantics  | This operator returns only the results from its argument that occur outside the given time window from the end of the file. |

---

## NOTLIKE

The following table provides information about the NOTLIKE operator:

| Info Type  | Description                                                                                                                                                                                                                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example    | prison = NOTLIKE("prison", "present", "president", "presumptive")                                                                                                                                                                                                                                         |
| Parameters | None                                                                                                                                                                                                                                                                                                      |
| Arguments  | The first argument may be any expression, but the second and all subsequent arguments must be phrases only (no operators). There must be at least two arguments. The second and subsequent arguments represent phonetically similar phrases which can be confused with the terms in the first expression. |

| Info Type | Description                                                                                                                                                                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Weights   | Supported                                                                                                                                                                                                                                                                                                 |
| Semantics | Lets you improve accuracy by explicitly identifying common similar-sounding phrases that can be confused with phrases in the first expression. The first argument is the target phrase to search for. The second and subsequent arguments represent potentially confusable phrases to your target phrase. |

---

## Sample Structured Queries

The sample queries in this topic demonstrate how to analyze large call center media files.

---

### Sample Structured Queries

#### Customer Service Script

```
; Script phrases
Time = Or("Good Morning.", "Good Afternoon.", "Good Evening.")
Greeting = Before_10(Time, "Thank you for calling Nexidia.")
Introduction = Before_10("My name is", "How may I help you?")
Help = "Let me help you with that"
OfferA = "Have you heard about our new"
Offer1 = "structured mining technology?"
Offer2 = "NEX Miner Pro product?"
Offer = Before_10("Have you heard about", Or(Offer1, Offer2))
RequestContinue = "Is there anything else I can do for you"
Closing = "Thank you calling Nexidia. Goodbye."
;
; Forbidden phrases
Bad1 = "Go away"
Bad2 = "Call back later"
;
; not allowed to say you don't know without
; also offering to find out
Bad3 = AndNot_10("I don't know", "I will find out for you")
```

#### Quality Monitoring

```
; This section can be added to the end of the Customer
; Service Script query above.
;
; Make sure the greeting is present
Initial = First_20(And_20(Greeting, Introduction))
```

```

;
; Make sure the closing is present
Ending = Last_20(And_20(RequestContinue, Closing))
;
;
; A good call has all of the above and is less than 10min
Good = And_600([1.0]Initial, [0.5]Help, [0.5]Offer, [1.0]Ending)
;
;
; Make sure they don't say anything they shouldn't
Result = AndNot_600(Good, Bad1, Bad2, Bad3)
;
;
; the calls we're concerned about are the ones that
; score poorly on this query (assumes < 10 min calls)

```

---

## References

- Berners-Lee, T., Fielding, R., and Masinter, L. Uniform Resource Identifier (URI): Generic Syntax. Available from <http://www.ietf.org/rfc/rfc3986.txt> (2005); accessed 25 August 2011.
- Fielding, R.T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine. Available from <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (2000); accessed 3 August 2010.
- Franks, J., Hallem-Baker, P., Hostetler, J., Lawrence, S., Leach, P., Luotonen, A., and Stewart, L. HTTP Authentication: Basic and Digest Access Authentication. Available from <http://www.ietf.org/rfc/rfc2617.txt> (1999); accessed 28 October 2011.
- Freed, N., Borenstein, N. Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies. Available from <http://www.ietf.org/rfc/rfc2045.txt> (1996); accessed 19 January 2012.
- Josefsson, S. The Base16, Base32, and Base64 Data Encodings. Available from <http://www.ietf.org/rfc/rfc4648.txt> (2006); accessed 25 March 2011.
- Risney, D. File URIs in Windows. Available from <http://blogs.msdn.com/b/ie/archive/2006/12/06/file-uris-in-windows.aspx> (2006); accessed 25 August 2011.
- W3C. Forms. In Ragget, D., Le Hors, A., and Jacobs, I. (Eds.), HTML 4.01 Specification. Available from <http://www.w3.org/TR/REC-html40/interact/forms.html> (1999); accessed 7 May 2012.